ESCUELA COLOMBIANA DE INGENIERÍA DESARROLLO ORIENTADO POR OBJETOS

Herencias e Implementaciones 2026-2 — Laboratorio 3/6

OBJETIVO

Evidenciar el logro de los siguientes resultados de aprendizaje:

1. Aprovechar los mecanismos de herencia e implementación.
2. Organizar las fuentes en paquetes.
3. Usar la utilidad jar de java para entregar una aplicación.
4. Extender una aplicación cumpliendo especificaciones de diseño, estándares y verificando su corrección.
5. Experimentar las prácticas XP:

   Coding. Code must be written to agreed standards.

   Testing. Code the unit test first.

ENTREGA

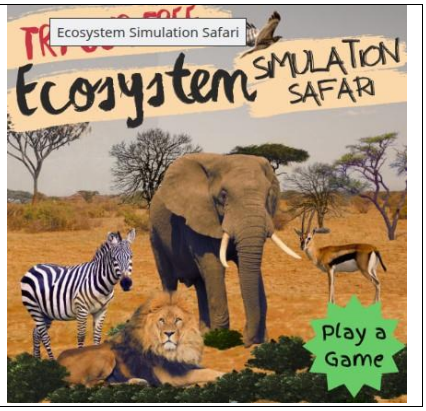
- Incluyan en un archivo .zip los archivos correspondientes al laboratorio. El nombre debe ser los apellidos de los dos miembros del equipo ordenados alfabéticamente y separados por un guion. (CasallasC-DuarteJ) Publiquen el avance al final de la sesión y la versión definitiva en la fecha indicada, en los espacios correspondientes.

CONTEXTO Reto: Desarrollar un simulador como **Ecosystem Safari Simulation**

Ecosystem Safari Simulation es un simulador desarrollado por *Conservation Mag*, una revista digital internacional enfocada en la conservación de la naturaleza. *Conservation Mag* tiene como objetivo principal de traducir la investigación científica compleja en artículos accesibles y atractivos para el público general.[SIMULADOR]

Nuestro simulador, denominado **EcoSafari**, representa una pradera africana en el que conviven animales, plantas y minerales. Todos representados en una cuadricula bidimensional. La pradera evoluciona mediante una secuencia de pasos discretos (tic-tac). En cada paso cada elemento actúa, siguiendo un orden fijo de izquierda a derecha y de norte a sur.

El **reto** es simular un modelo depredador-presa-recurso que describe ecosistemas en los que la supervivencia de una especie depende de otra.



PARTE I. Conociendo el proyecto

A Revisando línea base [En lab03.pdf]

1. En el directorio descarguen los archivos contenidos de safari.zip. Revisen el código:
   (a) ¿Cuántos paquetes tiene?
   Safari.zip tiene 1 paquete general llamado ecoSafari el cual contiene otros 3 paquetes: doc, domain, presentation.
   - Doc tiene los siguientes paquetes: aplicacion, domain, legal, presentacion, presentation, resources.
   - También tiene domain y presentation como paquetes.

   (b) ¿Cuál es el propósito del paquete presentación?
   Presentación tiene como proposito, contener la clase EcoSafariGUI, ofrecer la parte gráfica de todo el protecto tanto como la clase principal del dominio que es EcoSafari como su parte gráfica (librerías: awt, swing, io, awt.event).

   (c) ¿Cuál es el propósito del paquete dominio?
   Dominio tiene como propósito, contener EcoSafari, Elephant, Entity, Organism, este se caracteriza por manejar la lógica de negocio de este proyecto, a diferencia de el paquete presentation no se encarga de absolutamente nada gráfico, es la capa que si cambia se modifica la manera en la que reaccionan los objetos dentro de EcoSafaryGUI al pintarse.

2. Revisen el paquete de dominio,
   (a) ¿Cuáles son los diferentes tipos de componentes de este paquete?
   Los tipos de componentes de este paquete son:
   - Clases concretas (EcoSafari, EcoSafariGUI, Elepant)
   - Clase Abstracta (Organism)
   - Interfaz (Entity)

   (b) ¿Qué implica cada uno de estos tipos de componentes?

//////////////////////	Clase Concreta	Clase Abstracta	Interfaz
¿Se pueden tener atributos?	Sí, de todo tipo.	Sí de todo tipo.	Solamente atributos que tengan modificador de pertenencia y mutabilidad (static o final en Java).
¿Qué tipo de métodos puede tener?	Solamente concretos.	Concretos y abstractos.	Implementaciones por defecto o predeterminado (default) y abstractos.
¿Se puede instanciar?	Sí.	No.	No.

La herencia compromete mucho a un componente de tal manera que tiene que implementar toda su lógica abstracta y además de eso solamente le permite en Java tener una sola herencia, ahora cuando se implementa una interfaz es más flexible el comportamiento que aquel componente implementa de aquella interfaz o interfaces esto porque Java permite implementar varias interfaces.

3. Revisen el paquete de presentación,
   (a) ¿Cuántos componentes tiene?
   Tiene 1 solo componente EcoSafariGUI el cual es una clase concreta.

   (b) ¿Cuántos métodos públicos propios (no heredados) ofrece?
   Tiene 8 métodos propios (no heredados):
   - EcoSafariGUI (constructor)
   - PrepareElements
   - PrepareActions
   - TicTacButtonAction
   - GettheEcoSafari
   - main(String[] args)

   También tiene una clase interna PhotoEcoSafari que tiene los siguientes metodos:
   - PhotoEcoSafari(EcoSafariGUI gui)
   - paintComponent(Graphics g)

4. Para ejecutar un programa en java,
   (a) ¿Qué método se debe ejecutar?
   Se debe ejecutar el método main de la clase presentación (en el proyecto trabajado de este laboratorio).

   (b) ¿En qué clase se encuentra?
   Se encuentra en clase de presentación llamada EcoSafariGUI por lo que es la encargada de manejar el apartado gráfico del proyecto.
   Deben ejecutar la aplicación java, no crear un objeto como lo veníamos haciendo

5. Ejecuten el programa.
   (a) ¿Qué funcionalidades ofrece?
   El programa ofrece:
   - Botón Tic-tac: no ofrece funcionalidad.
   - Botón de eliminar y minimizar pestaña.

   (b) ¿Qué hace actualmente?
   Actualmente no hace nada el programa más que abrir una pestaña con una cuadricula.

   (c) ¿Por qué?
   Esto se debe a que no se implementó el método tic-tac en el dominio en la clase EcoSafari.

```java
/**
 * Advances the simulation by one time step
 */
//First, all entities execute their tic() action
//Then, all entities execute their tac() actions
public void ticTac(){
}
```

B Realizando ingeniería inversa. Arquitectura General [En lab03.pdf safari.astah]

1. Consulten el significado de las palabras package e import de java.

(a) ¿Qué es un paquete?

Un paquete en Java es un contenedor que agrupa componentes de software (Clases; abstractas o concretas. Interfaces).

(b) ¿Para qué sirve?

La función principal de un paquete en Java es agrupar los componentes, contener la lógica de secciones de la aplicación para la seguridad; poder extender el proyecto de forma segura y ordenada.

(c) ¿Para qué se importa?

Los paquetes se importan para que otros paquetes puedan acceder a su contenido, relacionarse con otros objetos o clases; usar funcionalidades de diferentes clases en el proyecto que se maneje.

(d) Expliquen su uso en este programa.

Su utilidad es separar la presentación del dominio con el fin de separar responsabilidades y reducir problemas de implementación. La clase presentación llama a la clase dominio para utilizarla, lo cual se hace de igual forma como si fuera una librería.

```
package presentation;
import domain.*;

import javax.swing.*;
import java.awt.*;
import java.awt.event.*;
import java.io.*;
```

2. Revisen el contenido del directorio de trabajo y sus subdirectorios. Describan su contenido.

Contenido:

- Presentacion:
  - EcoSafariGUI$1.class
  - EcoSafariGUI$PhotoEcoSafari.class
  - EcoSafariGUI.class
  - EcoSafariGUI.ctxt
  - EcoSafariGUI.java
  - Package.bluej
  - PhotoEcoSafari.class
  - PhotoForest.class
  - PhotoValley.class
- Domain
  - EcoSafari.class
  - EcoSafari.ctxt
  - EcoSafari.java
  - Elephant.class
  - Elephant.ctxt

- Elephant.java
- Entity.class
- Entity.ctxt
- Entity.java
- Organism.class
- Organism.ctxt
- Organism.java
- Package.bluej

- Doc
  - allclasses-frame.html
  - allclasses-noframe.html
  - constant-values.html
  - Copy.svg
  - element-list
  - help-doc.html
  - Index.html
  - index-all.html
  - Link.svg
  - Logfile.txt
  - overview-frame.html
  - overview-summary.html
  - overview-tree.html
  - package-list
  - Script.js
  - serialized-form.html
  - Stylesheet.css
  - Aplicacion
    1) Celula.html
    2) package-frame.html
    3) package-summary.html
    4) package-tree.html
  - Domain
    1) Organism.html
    2) package-frame.html
    3) package-summary.html
    4) package-tree.html
  - Legal
    1) ADDITIONAL_LICENSE_INFO
    2) ASSEMBLY_EXCEPTION
    3) LICENSE
  - Presentacion
    1. package-frame.html
    2. package-summary
    3. package-tree.html
  - Presentation
    1) package-frame.html

2) package-summary.html
3) package-tree.html
4) PhotoAManufacturing.html
5) PhotoCity.html

- Resources
  1) inherit.gif

La agrupación de extensiones de:

- .class
- .ctxt
- .java

Hacen referencia a todos los archivos necesarios para que una clase de java funcione en BlueJ.

- .class: es el bytecode de la máquina virtual de java.
- .ctxt: es la configuración de BlueJ para la interfaz gráfica.
- .java: es el código el lenguaje entendible por un humano.
- .bluej: archivo de BlueJ

(a) ¿Qué coincidencia hay entre paquetes y directorios?

Cada directorio representa un paquete, el directorio dominio hace referencia al paquete dominio y contiene sus clases, de igual forma para el maquete y directorio presentación.

3. Adicionen al diseño la arquitectura general con un diagrama de paquetes en el que se presente los paquetes y las relaciones entre ellos. Consulte la referencia en Moodle. En astah, crear un diagrama de clases (cambiar el nombre por Package Diagram0)

Diagrama de clase de Dominio:

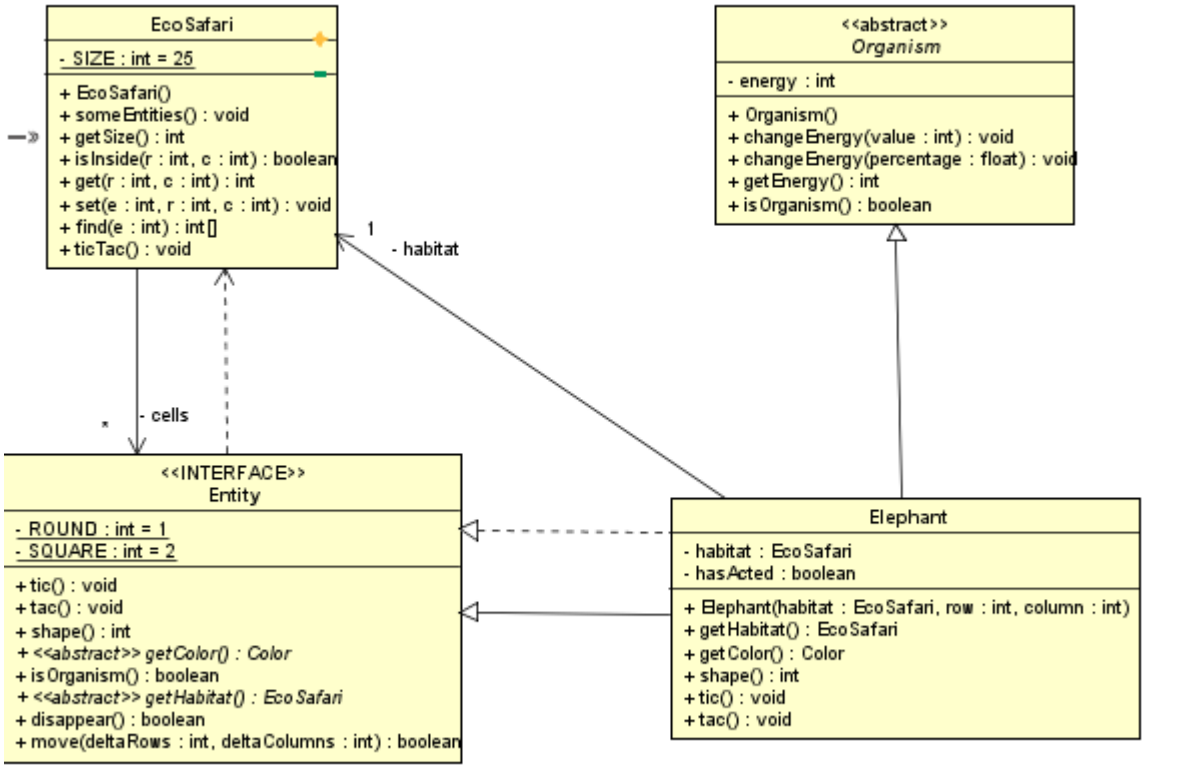
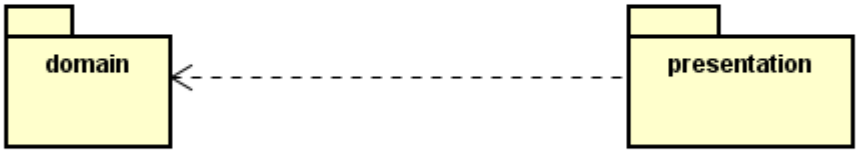


Diagrama de la clase de presentation:



Diagrama de paquetes:


domain
presentation

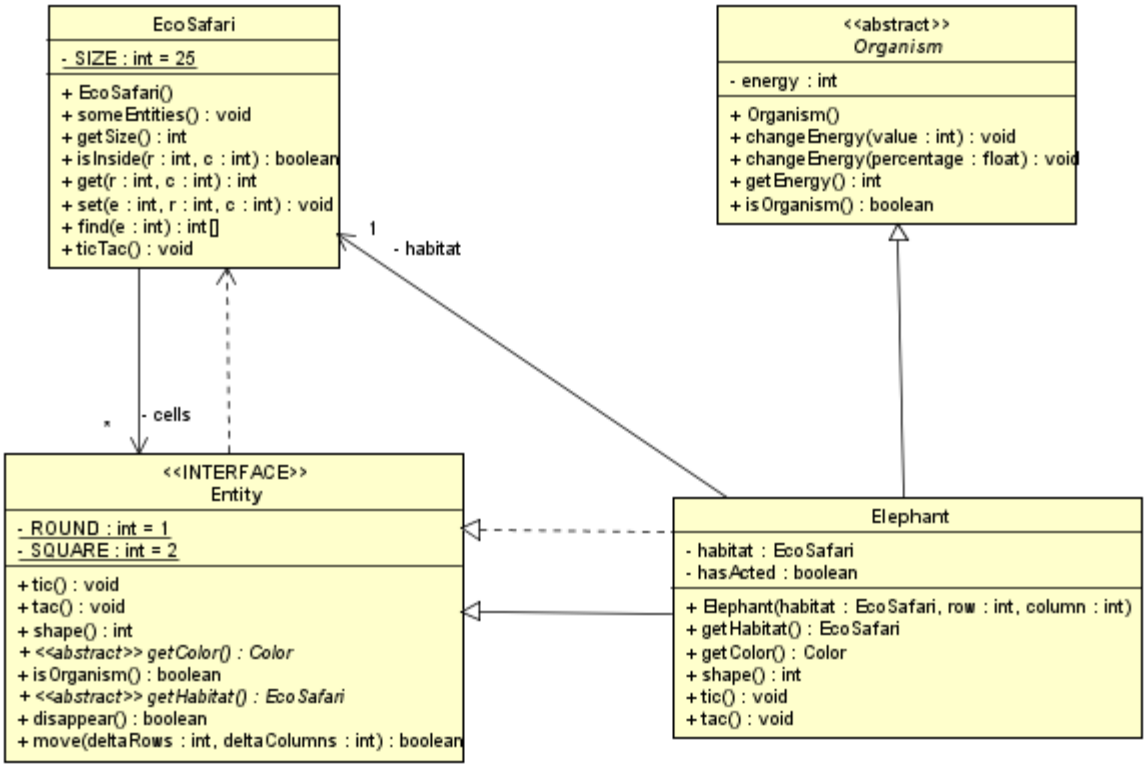
C Realizando ingeniería inversa. Arquitectura detallada [En lab03.pdf safari.astah]

1. Para preparar el proyecto para MDD. Realicen el diseño detallado del paquete de dominio.

(a) ¿Qué componentes hacían falta?

Los componentes que hacian faltan eran:

1) Todo el diagrama de clase.
2) La clase Elephant.
3) La clase Entity.



2. Realicen el diseño detallado del paquete de presentación.

a) ¿Por qué hay dos clases y un archivo .java?

Porque dentro de la clase EcoSafariGUI, se declara otra clase llamada PhotoEcoSafari, entonces en realidad solo existe un archivo .java ya que solo se declara una clase de forma general.



3. Adicione la clase de pruebas necesaria para BDD en un paquete independiente de test. (No lo adicione al diagrama de clases)

(a) ¿Qué clase debe importar?

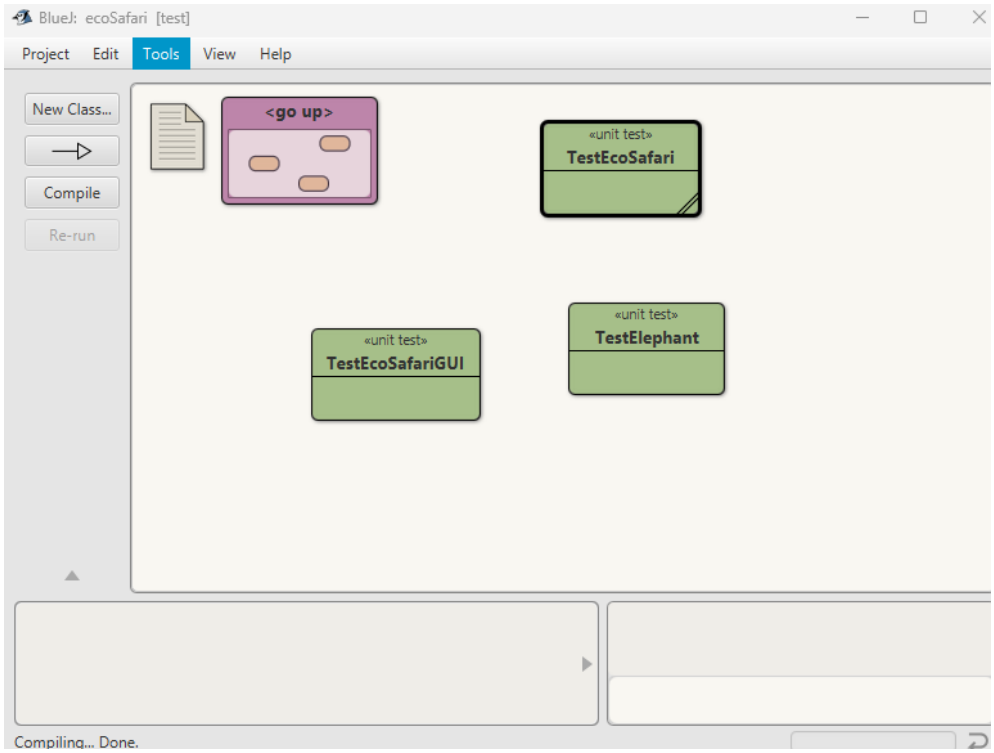
Se deben importar las clases que se van a utilizar para las pruebas:

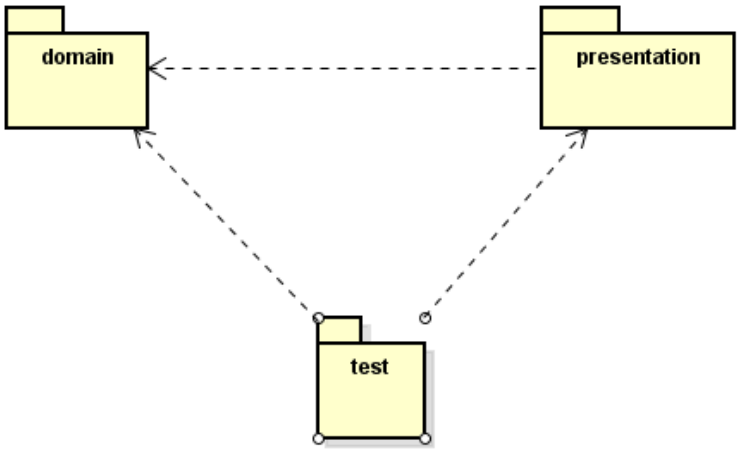
- EcoSafariGUI.
- Elephant.
- EcoSafari.

Se importan en cada prueba relacionada a cada clase

(b) ¿Por qué?

Porque son las clases que vamos a programar gradualmente y la que determinan si la lógica está o no funcionando en el programa desde lo pequeño, hasta la integración total con EcoSafariGUI.


BlueJ: ecoSafari [test]
Project
Edit
Tools
View
Help
New Class...
Compile
Re-run
<go up>
«unit test»
TestEcoSafari
«unit test»
TestElephant
«unit test»
TestEcoSafariGUI
Compiling... Done.


domain
presentation
test

PARTE II. Desarrollando el proyecto

Para desarrollar esta aplicación vamos a considerar algunos ciclos.

A Ciclo 1. Iniciando con elefantes [En lab03.pdf safari.astah *.java]

(NO OLVIDE BDD – MDD)

1. Estudie la clase EcoSafari .

(a) ¿Qué tipo de colección usa para albergar cosas?

La colección que utiliza EcoSafari para albergar cosas es una lista de listas que contiene Entity

(b)¿Puede tener elefantes?

Si puede tener elefantes por medio de la interfaz Entity

(c) ¿Por qué?

Ya que elefantes implementa la interfaz y EcoSafari tiene Entity, en consecuencia EcoSafari puede llegar a tener elefantes.

2. Estudie el código asociado a la clase Elephant ,

(a) ¿de qué color es?

El color del elefante varía según el nivel de energía:

- Gris Oscuro: Si la energía es mayor o igual a 80.
- Gris Claro: Si la energía es menor a 80.

(b) ¿qué forma usa para pintarse?

Para pintarse, el elefante utiliza la libreria java.awt.Color y se pinta utilizando Color.DARK_GRAY y Color.LIGHT_GRAY.

(c) ¿cómo se mueve?

El elefante se mueve llamando al método move(1, 1) dentro de su método tic(), le resta energía por cada movimiento y si la energía es igual a 0 lo desaparece.

(d) ¿qué componentes definen la clase Elephant ? Justifique sus respuestas.

Los componentes que definen la clase Elephant, es una herencia que tiene con Organism y implementa una interfaz, la cual es Enity.

3. Elephant por ser un Organism ,

(a) ¿qué atributos tiene?

Tiene los siguientes atributos:

```
//Include the documentation
public class Elephant extends Organism implements Entity{
    private final EcoSafari habitat;
    private boolean hasActed;
```

Y este:

```
public abstract class Organism{

    private int energy; //Inv: 0<=energy<=100
```

(b) ¿qué puede hacer (métodos)?

Con el Elefante se puede crear, obtener su habitad, su color, su figura, hacerlo mover o reaccionar segun sea tic o tac el momento del tablero junto a sus comportamientos de su superclase; cambiar de energía, dar la energía actual que tiene y responder a si es un organismo.

(c) ¿qué decide hacer distinto (sobre-escritura)?

Lo que piensa hacer distinto es en el momento tac y tic hacer su movimiento en específico, también obtener el color característico según su energía y obtener su habitad en específico.

(d) ¿qué no puede hacer distinto (métodos finales)?

No puede cambiar de energia, ni tampoco responder si es un organismo porque ya está definido, ni tampoco tener otra forma de brindar la eneria actual que tiene.

(e) ¿qué debe aprender a hacer (métodos abstractos)? Justifique sus respuestas.

Tiene que aprender a hacer el momento tic del tablero, el obtener su color especifico y saber obtener su habitad en específico. Esto tiene sentido en su manera que brinda información a presentación para poder pintarse.

4. Por comportarse como un Entity ,

(a) ¿qué debe hacer?

Tiene que aprender a hacer el momento tic del tablero, el obtener su color especifico y saber obtener su habitad en específico.

(b) ¿qué decide hacer distinto?

Se decide por brindar una figura del elefante en específico y además de actuar diferente en el momento tac.

(c) ¿qué no puede hacer distinto?

No puede cambiar de energía, ni tampoco responder si es un organismo porque ya está definido, ni tampoco tener otra forma de brindar la energía actual que tiene.

(d) ¿qué debe aprender a hacer? Justifique sus respuestas.

Saber obtener el habitad en específico y obtener su color en específico.

5. De acuerdo a lo anterior un Elephant ,

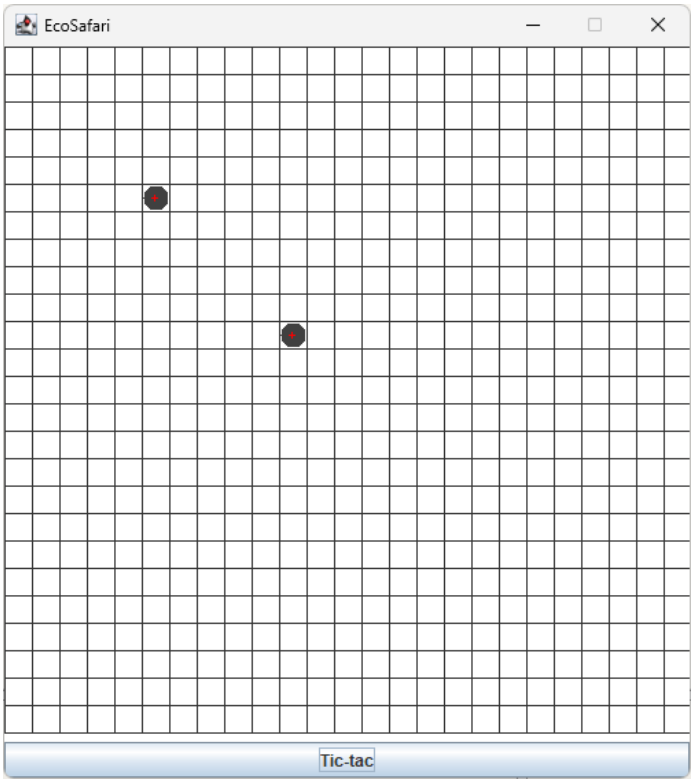
(a) ¿Cómo actúa?

6. Ahora vamos a crear dos elefantes en diferentes posiciones (5,5) (10,10) , llámelos dumbo y babar , usando el método someEntities . Ejecuten el programa,

Se usa el método someEntities como se muestra a continuación

```java
/**
 * Pupulates the EcoSafari with some entities
 */
public void someEntities(){
    Elephant dumbo = new Elephant(this, 5, 5);
    Elephant babar = new Elephant(this, 10, 10);
}
```

Ya instanciando la interfaz gráfica de presentación se tiene el siguiente resultado, se pintan dos elefantes en las posiciones (5, 5) y (10, 10)



(a) ¿Qué pasa con los elefantes?

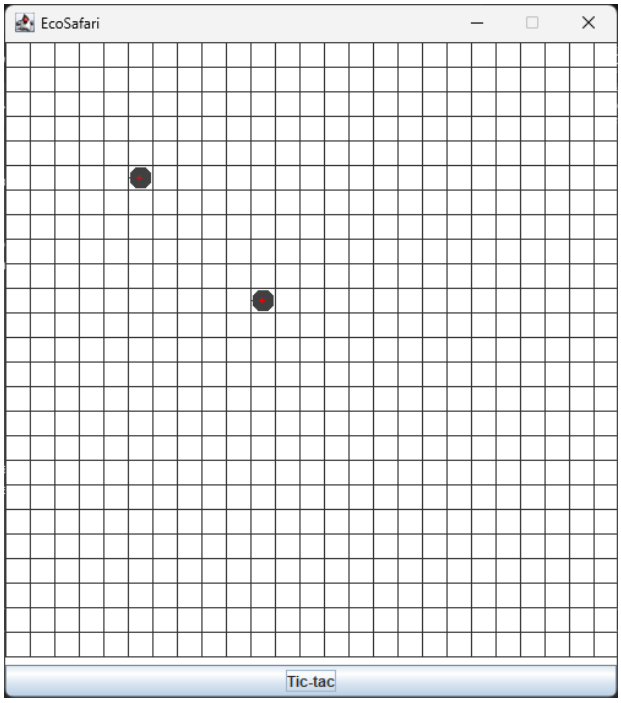
(b)¿Por qué?

Ahora se va a presionar el botón Tic-tac para ver su funcionamiento, más al presionar varias veces no se mueven los elefantes, esto debido a que no se ha implementado el método ticTac de EcoSafari como se muestra a continuación

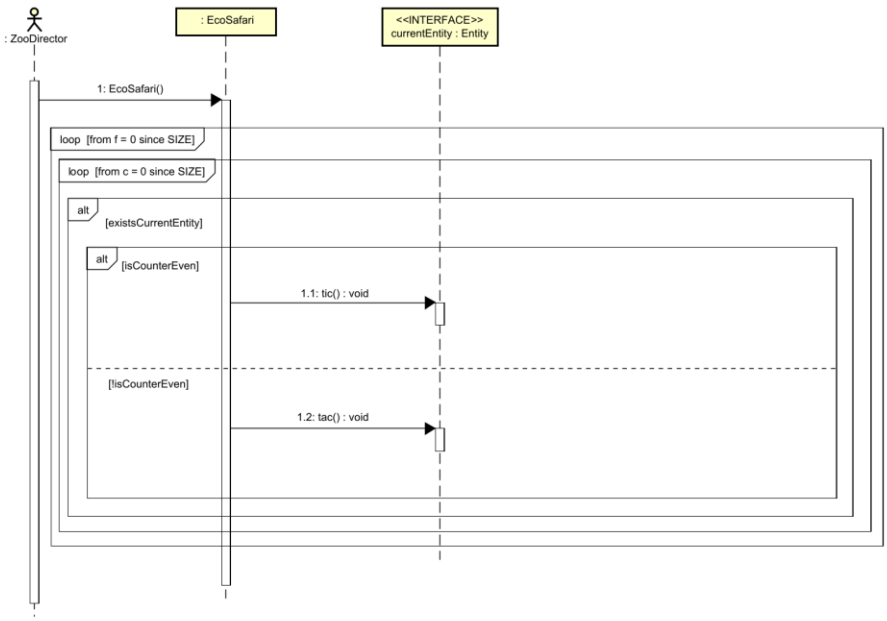
```java
/**
 * Advances the simulation by one time step
 */
//First, all entities execute their tic() action
//Then, all entities execute their tac() actions
public void ticTac(){
}
```

(c) Capturen una pantalla significativa.

En lo siguiente se muestra que siguen en la misma posición



7. Diseñen, construyan y prueben el método llamado ticTac() de la clase EcoSafari .



Ahora en la implementación del código se tiene lo siguiente

```java
/**
 * Advances the simulation by one time step
 */
public void ticTac(){
    boolean isCounterEven = false, existsCurrentEntity = false;
    Entity currentEntity = null;
    for (int f = 0; f<SIZE; f++) {
        for (int c = 0; c<SIZE; c++) {
            isCounterEven = counterTicTac%2==0;
            currentEntity = cells[f][c];
            existsCurrentEntity = currentEntity != null;
            if (existsCurrentEntity) {
                if (isCounterEven) {//First, all entities execute their tic() action
                    currentEntity.tic();
                } else if (!isCounterEven) {//Then, all entities execute their tac() actions
                    currentEntity.tac();
                }
            }
        }
    }
    counterTicTac++;
}
```

Su prueba es la siguiente

```java
public class TestEcoSafari {

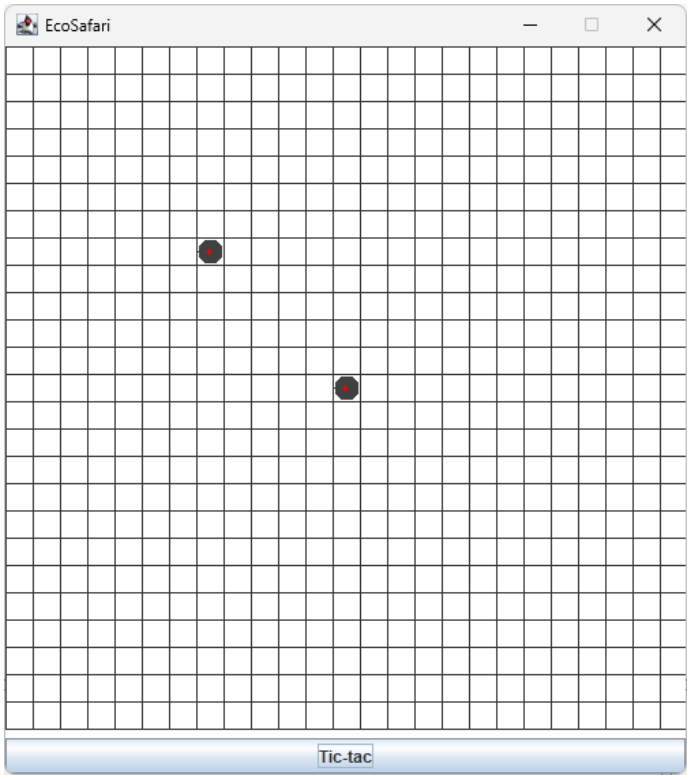
    private EcoSafari game;
    private Elephant dumbo;
    private Elephant babar;

    @BeforeEach
    public void setUp() {
        game = new EcoSafari();
        dumbo = (Elephant) game.get(5, 5);
        babar = (Elephant) game.get(10, 10);
    }

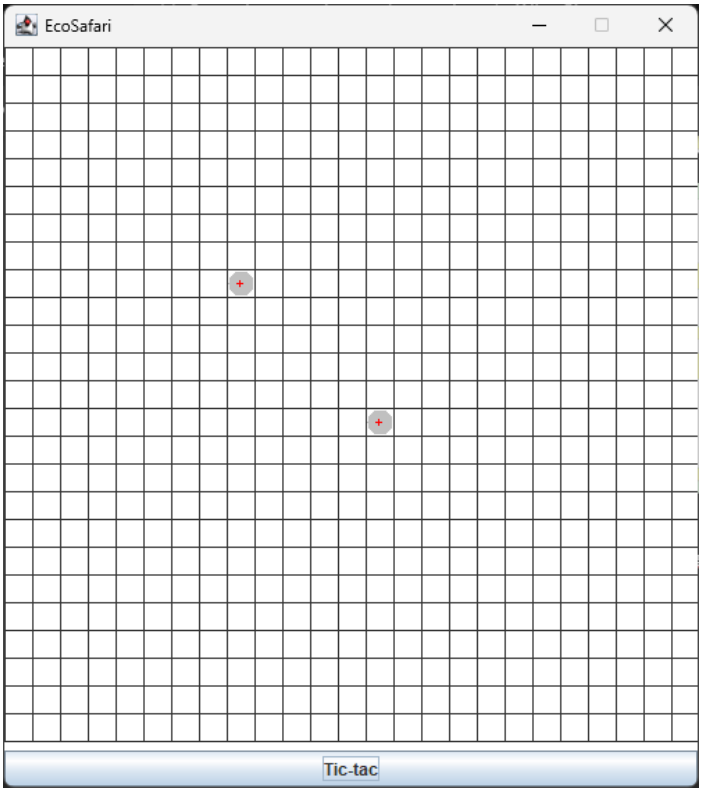
    @Test
    public void shouldTicTac() {
        int[] initialPosDumbo = game.find(dumbo);
        int[] initialPosBabar = game.find(babar);
        game.ticTac();
        int[] nextPosDumbo = game.find(dumbo);
        int[] nextPosBabar = game.find(babar);
        assertFalse(Arrays.equals(initialPosDumbo, nextPosDumbo)); // No deberian estar en la misma posicion
        assertFalse(Arrays.equals(initialPosBabar, nextPosBabar));
    }
}
```

8. (a) ¿Cómo quedarían dumbo y babar después de 3, 5, 10 tic-tac? Ejecuten el programa. Capturen pantallas significativas en los momentos correspondientes.

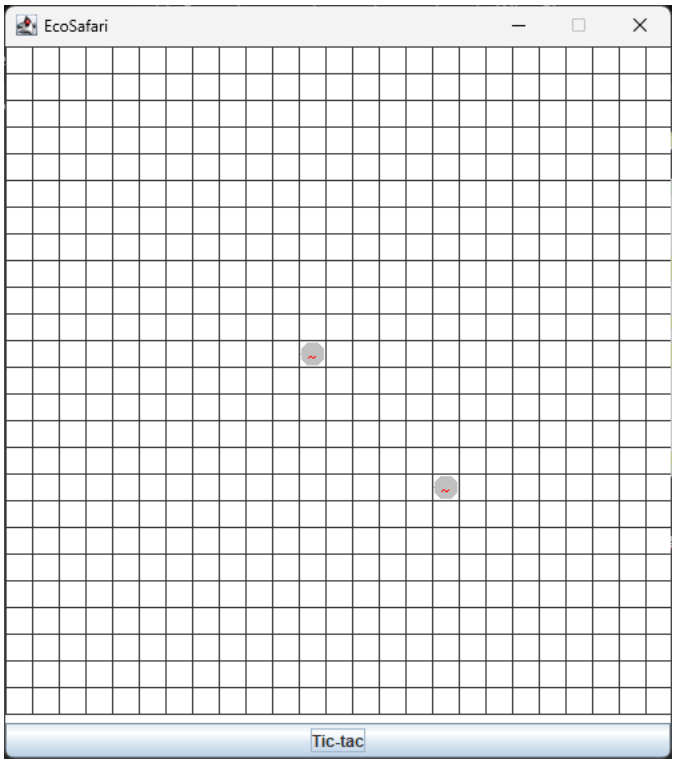
Cuando se le da 3 veces al botón tic-tac:



Dandole click 5 veces:



Con 10 veces presionado el botón



(b) ¿Es correcto?

Si, es correcto porque según el comportamiento del elefante debería cambiar su color cuando tiene menos de 80 de energía, además que se debe mover 1 casilla a su alrededor cada dos veces que se oprime el botón Tic-tac.

B. Ciclo 2. Incluyendo arbustos [En lab03.pdf safari.astah *.java]

(NO OLVIDE BDD – MDD)

El objetivo de este punto es permitir tener arbustos en el bosque. Ellos:

(i) son verdes, cuando son jóvenes y amarillos al envejecer (4 tics)

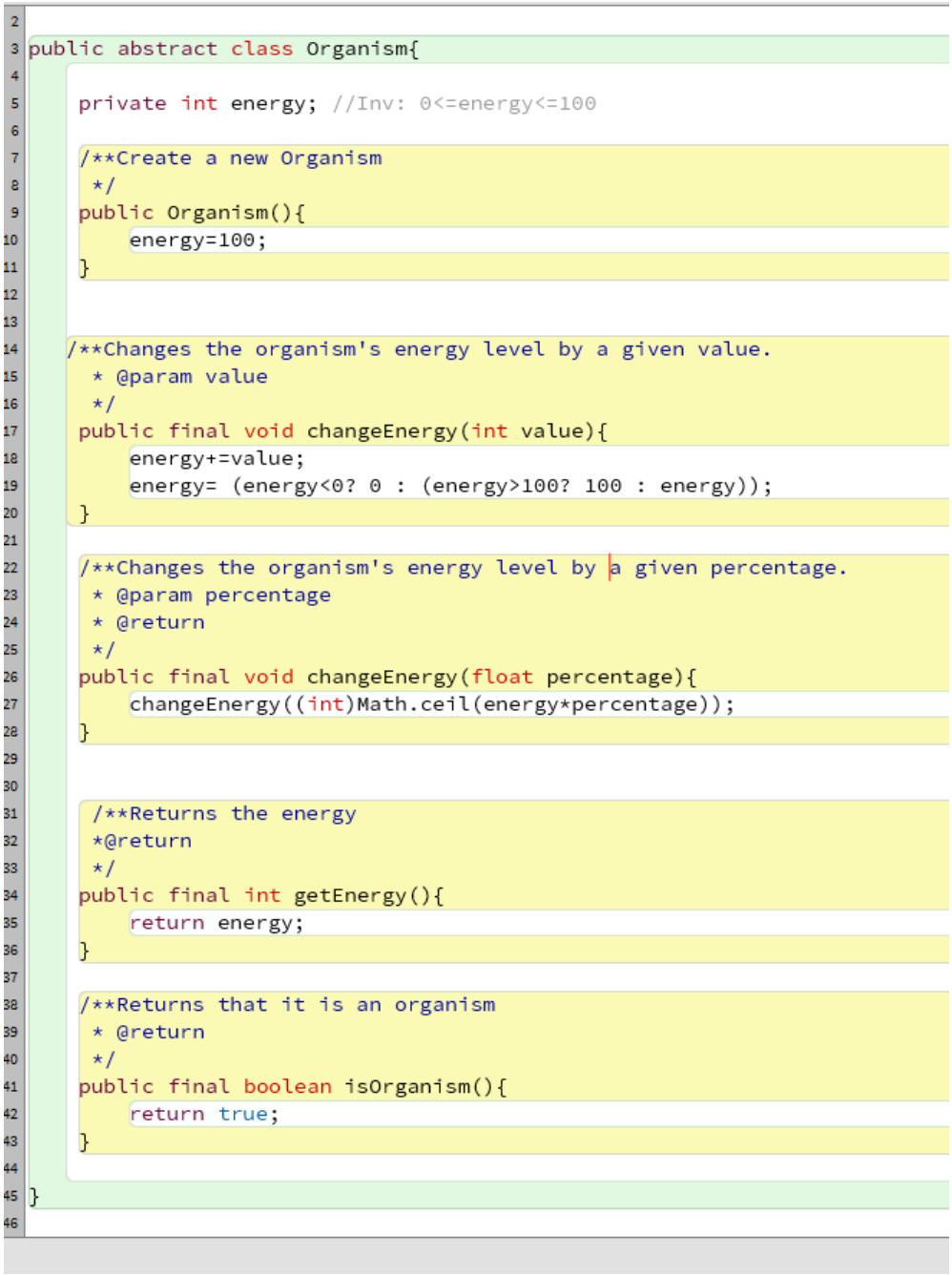
(ii) cuando son jóvenes, después de 2 tics retornan si hay espacio vecino vacío con el siguiente orden de prioridad: norte, sur, este y oeste,

(iv) si quedan cerca a un elefante desaparecen (el elefante se los come)

1. Para implementar esta nueva clase Organism

(a) ¿cuáles métodos se sobre-escriben (overriding)?

Ninguno de los métodos se puede sobre escribir dado a que todos los métodos inicialmente presentados tienen "final" lo que impide la sobre escritura.



```java
public abstract class Organism{

    private int energy; //Inv: 0<=energy<=100

    /**Create a new Organism
     */
    public Organism(){
        energy=100;
    }


   /**Changes the organism's energy level by a given value.
     * @param value
     */
    public final void changeEnergy(int value){
        energy+=value;
        energy= (energy<0? 0 : (energy>100? 100 : energy));
    }

    /**Changes the organism's energy level by a given percentage.
     * @param percentage
     * @return
     */
    public final void changeEnergy(float percentage){
        changeEnergy((int)Math.ceil(energy*percentage));
    }


     /**Returns the energy
     *@return
     */
    public final int getEnergy(){
        return energy;
    }

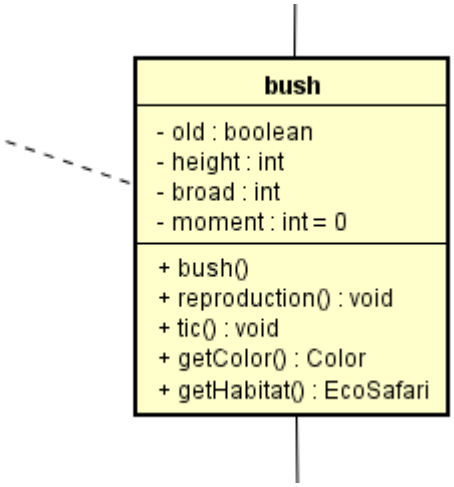
    /**Returns that it is an organism
     * @return
     */
    public final boolean isOrganism(){
        return true;
    }

}

```

2. Diseñen, construyan y prueben esta nueva clase. (Mínimo dos pruebas de unidad)

Arbusto es hijo de bush e implementa entity y usa EcoSafari, tambien tiene su metodo de reproducción y un atributo que nos indica si es o no viejo.



El esquema general del codigo es:

```java
package domain;
import java.awt.Color;


/**
 * Write a description of class bush here.
 *
 * @author (your name)
 * @version (a version number or a date)
 */
public class Bush extends Organism implements Entity
{
    private boolean old;

    public Bush() {

    }

    public void reproduction(){

    }

    @Override
    public void tic() {

    }

    @Override
    public Color getColor() {
        return null;
    }

    @Override
    public EcoSafari getHabitat() {
        return null;
    }
}
```

Las pruebas se diseñarán segun los 3 requerimientos planteados:

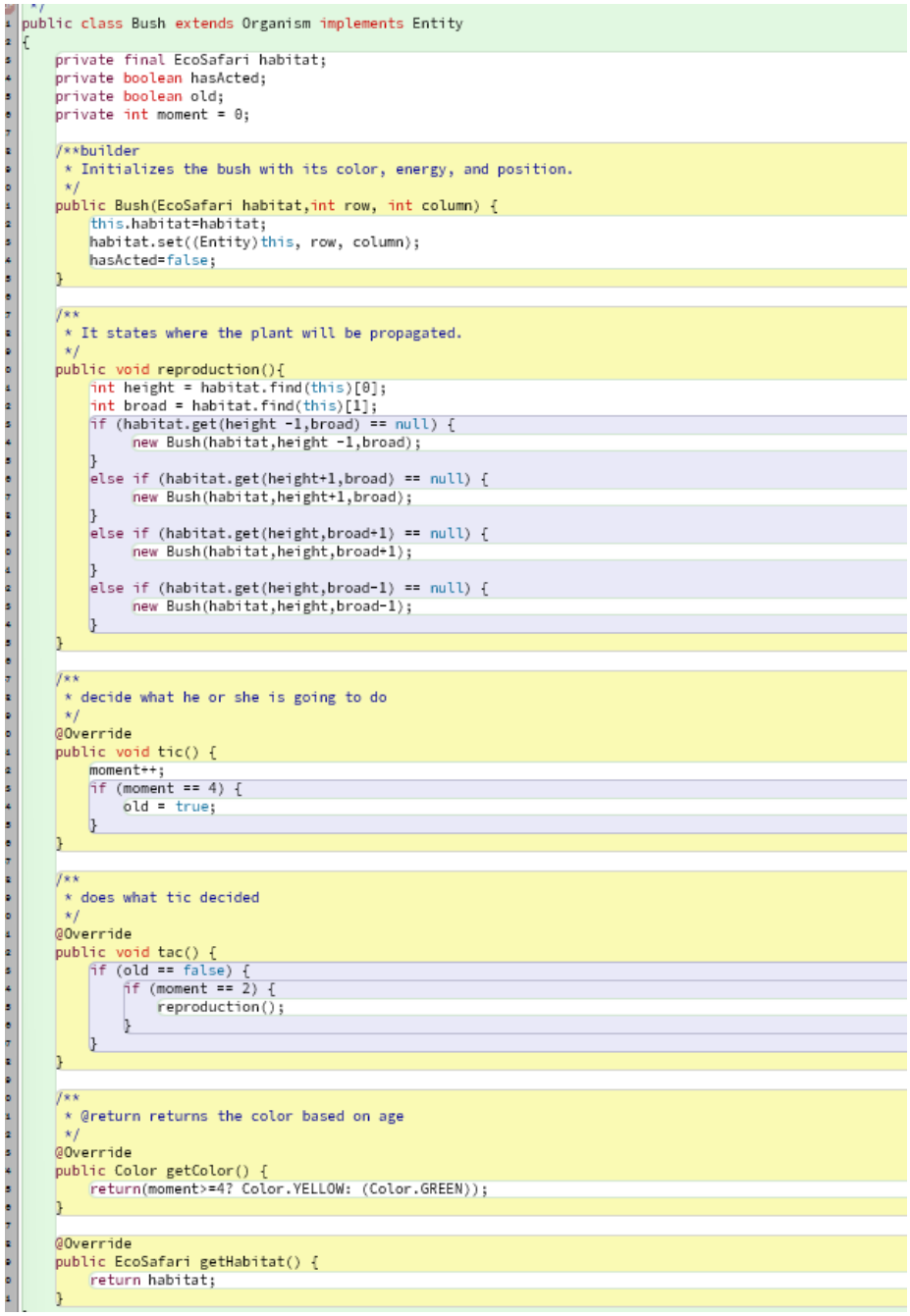
```java
import static org.junit.jupiter.api.Assertions.*;
import org.junit.jupiter.api.AfterEach;
import org.junit.jupiter.api.BeforeEach;
import org.junit.jupiter.api.Test;
import domain.Bush;
import domain.EcoSafari;
import domain.Elephant;
/**
 * The test class TestBush.
 *
 * @author  (your name)
 * @version (a version number or a date)
 */
public class TestBush
{
    private Bush bush;
    @Test
    public void shouldAge() {
        bush = new Bush();
        assertEquals(Color.GREEN , bush.getColor());
        bush.tic();
        bush.tic();
        bush.tic();
        bush.tic();
        assertEquals(Color.YELLOW , bush.getColor());
    }

    @Test
    public void shouldReproduction(){
        EcoSafari ecoSafari = new EcoSafari();
        bush = new Bush();
        ecoSafari.set(bush,5,5);
        bush.tic();
        bush.tic();
        assertTrue(ecoSafari.get(4,5) instanceof Bush);
    }

    @Test
    public void shouldDead() {
    EcoSafari ecoSafari = new EcoSafari();
    bush = new Bush();
    ecoSafari.set(bush,5,5);
    Elephant elephant = new Elephant(ecoSafari,4,5);
    bush.tic();
    elephant.tic();
    assertTrue( !(ecoSafari.get(4,5) instanceof Bush));
    }

}
```

Ahora hacemos los diagramas de flujo y programamos:



```java
*/
public class Bush extends Organism implements Entity
{
    private final EcoSafari habitat;
    private boolean hasActed;
    private boolean old;
    private int moment = 0;

    /**builder
     * Initializes the bush with its color, energy, and position.
     */
    public Bush(EcoSafari habitat,int row, int column) {
        this.habitat=habitat;
        habitat.set((Entity)this, row, column);
        hasActed=false;
    }

    /**
     * It states where the plant will be propagated.
     */
    public void reproduction(){
        int height = habitat.find(this)[0];
        int broad = habitat.find(this)[1];
        if (habitat.get(height -1,broad) == null) {
             new Bush(habitat,height -1,broad);
        }
        else if (habitat.get(height+1,broad) == null) {
             new Bush(habitat,height+1,broad);
        }
        else if (habitat.get(height,broad+1) == null) {
             new Bush(habitat,height,broad+1);
        }
        else if (habitat.get(height,broad-1) == null) {
             new Bush(habitat,height,broad-1);
        }
    }

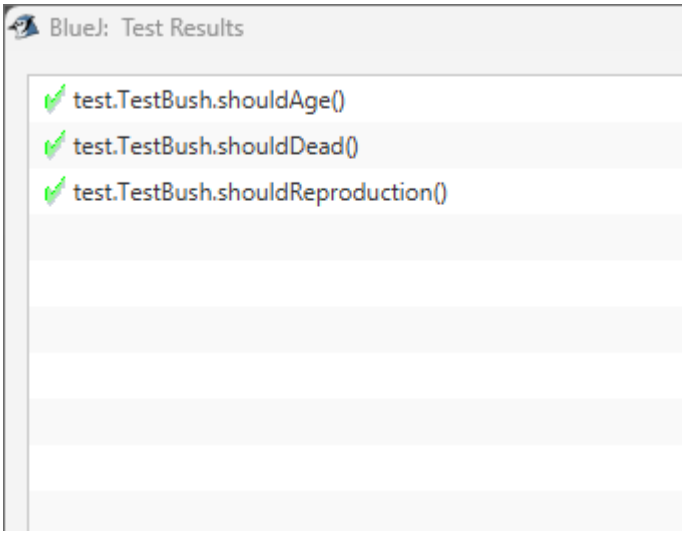
    /**
     * decide what he or she is going to do
     */
    @Override
    public void tic() {
        moment++;
        if (moment == 4) {
            old = true;
        }
    }

    /**
     * does what tic decided
     */
    @Override
    public void tac() {
        if (old == false) {
            if (moment == 2) {
                reproduction();
            }
        }
    }

    /**
     * @return returns the color based on age
     */
    @Override
    public Color getColor() {
        return(moment>=4? Color.YELLOW: (Color.GREEN));
    }

    @Override
    public EcoSafari getHabitat() {
        return habitat;
    }
}
```

Verificamos las pruebas:



3. Adicione una pareja de arbustos , de nombre mopane y acacia ,

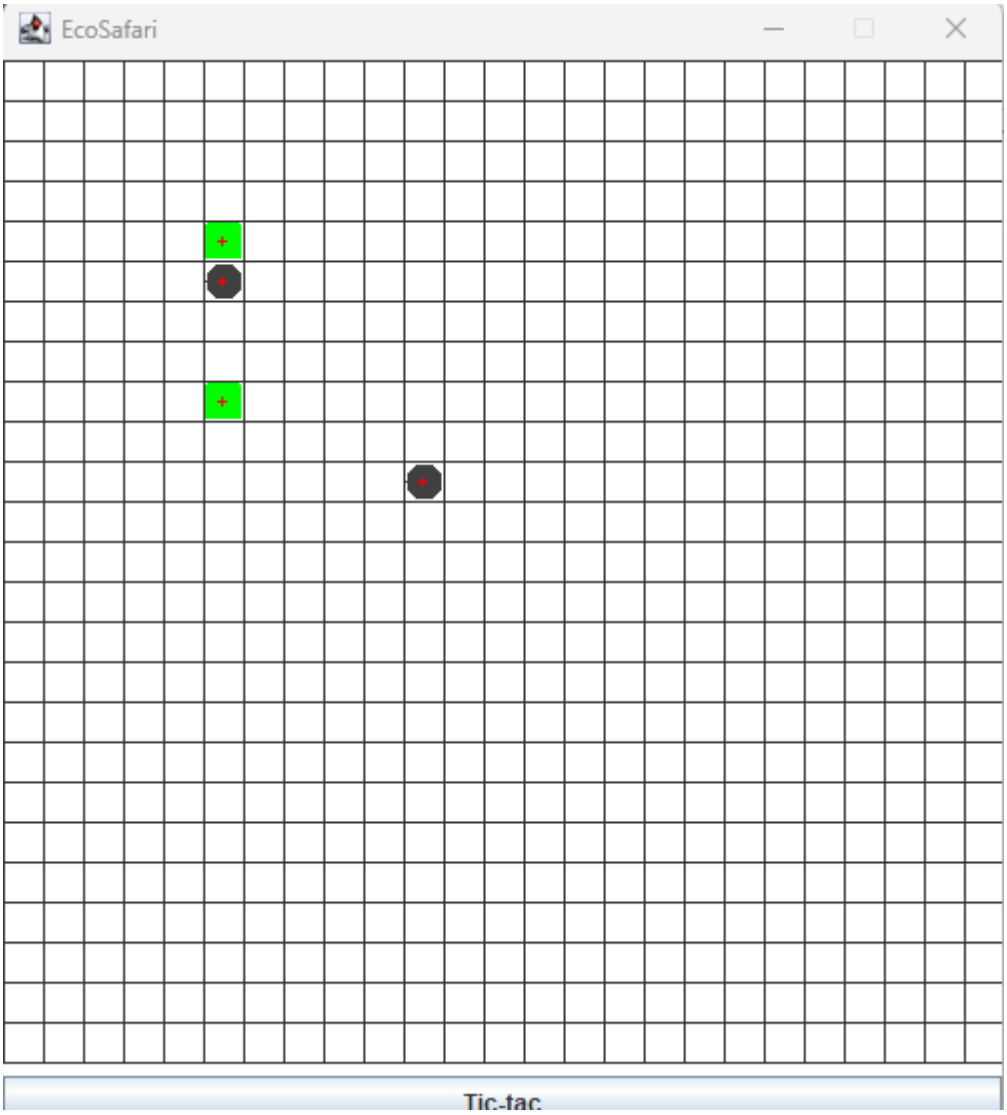
(a) ¿Cómo quedarían después de 3 Tic-tac? Ejecuten el programa y hagan 3 clics en el botón. Capturen una pantalla significativa.

Adicion:

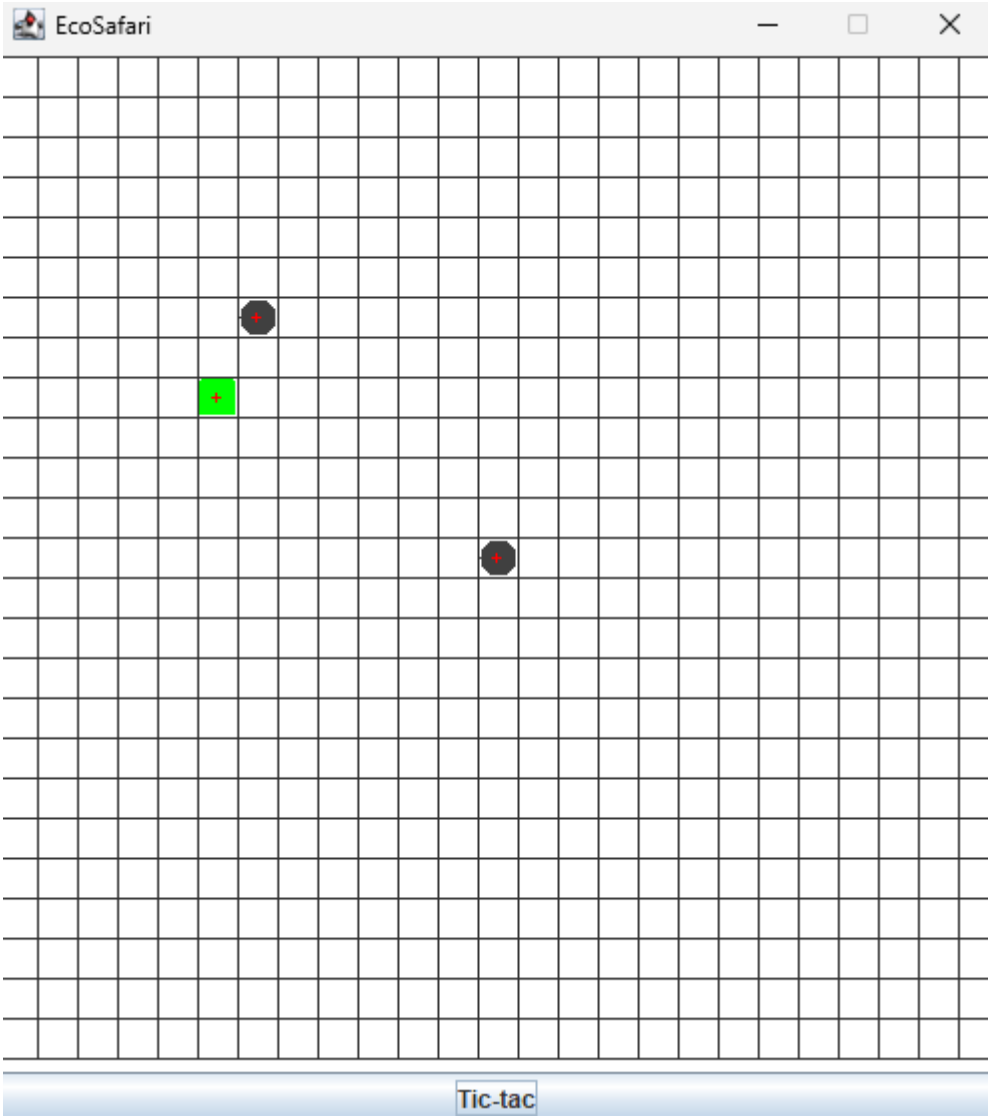
```
/**
 * Pupulates the EcoSafari with some entities
 */
public void someEntities(){
    Elephant dumbo = new Elephant(this, 5, 5);
    Elephant babar = new Elephant(this, 10, 10);
    Bush mopane  = new Bush(this, 4, 5);
    Bush acacia  = new Bush(this, 6, 5);
}

/**
 * Returns the size of the EcoSafari
```

Antes de ejecutar los 3 Tic-tac:



Despues:



(b) ¿Es correcto?
Si, cumple con la logica

Nota: para que siga cumpliendo las pruebas unitarias, se cambió la posición de los arbustos después de la muestra en él informa del laboratorio.

C. Ciclo 3. Adicionando tormentas [En lab03.pdf safari.astah *.java]

(NO OLVIDE BDD – MDD)

El objetivo de este punto es incluir tormentas (sólo vamos a permitir el tipo básico). En una tormenta (i) su centro se desplaza circularmente en diagonal en dirección noreste, (ii) se afecta un área diámetro tres, (iii) destruye lo que encuentra en su centro, (iv) su centro es negro y vuelve gris el área afectada

1. Para poder adicionar tormentas ,

(a) ¿debe cambiar en el código de EcoSafari en algo?

En sí de la clase EcoSafari no debería cambiar nada, pero sí el código del proyecto se debe extender

(b) ¿por qué?

Porque una tormenta no es un organismo vivo, entonces se debe hacer la interfaz

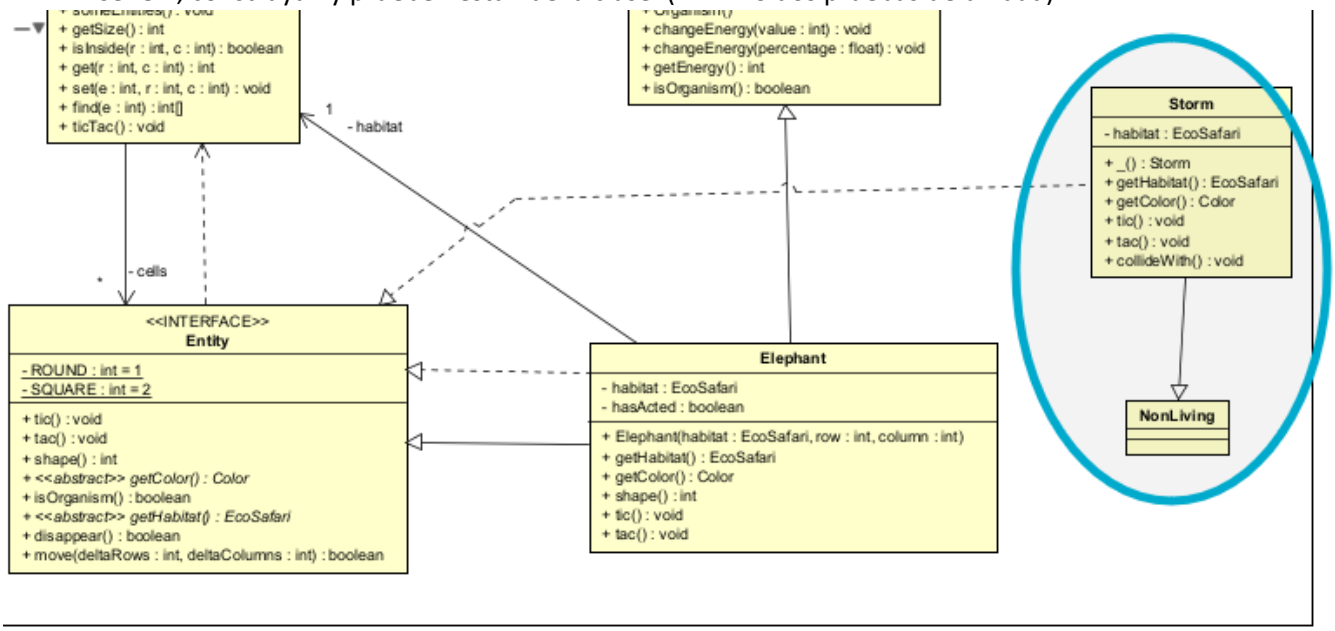
(c) ¿debe extender Entity ?

Si, sin embargo, surge el problema en que como se hace un método para cada momento del botón tictac, un tic() y un tac() entonces surge lo complejo en el aspecto de que la tormenta.

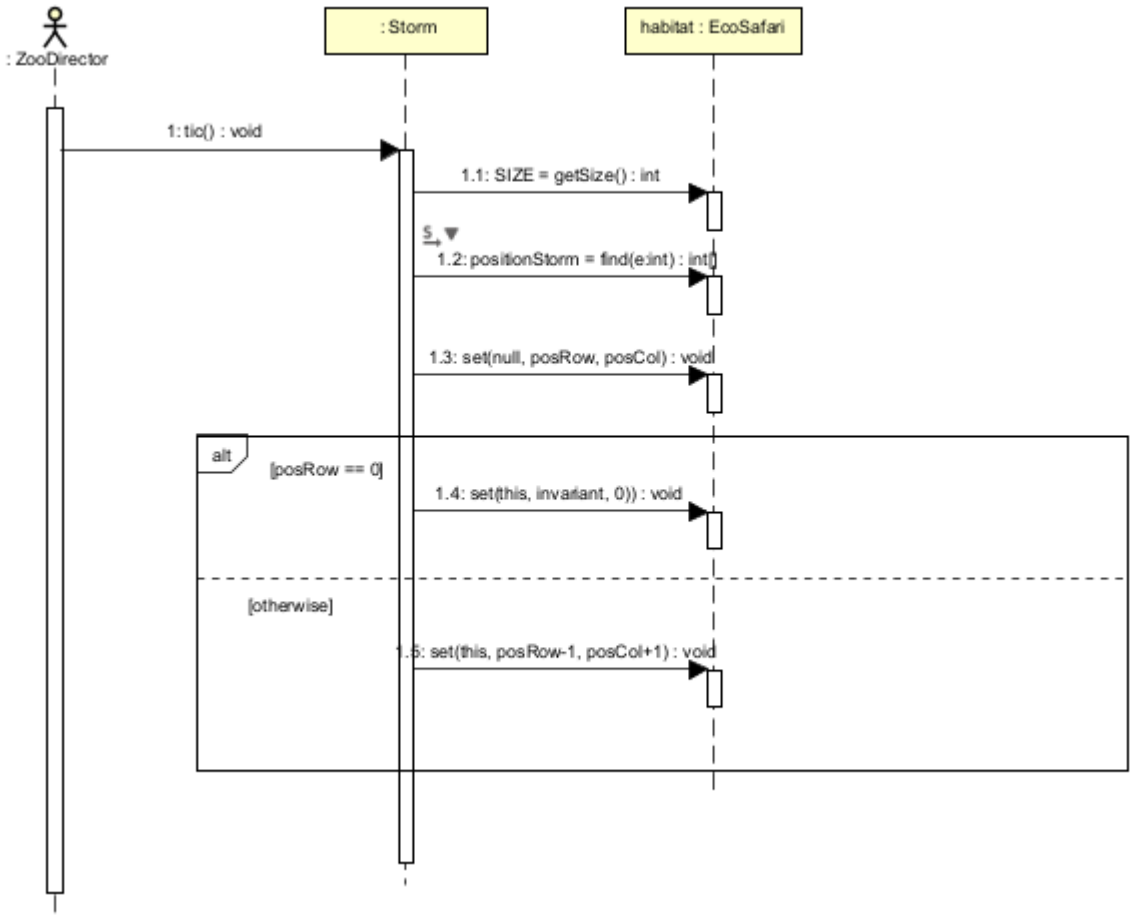
(d) ¿por qué?

Por como es el requisito de Strom que requiere que en cada movimiento se mueva, esto hace que se repita código en tic() y tac().

2. Diseñen , construyan y prueben esta nueva clase. (Mínimo dos pruebas de unidad)



El diagrama de secuencia más importante que se hizo es



Y probando esta funcionalidad se tienen lo siguientes test



Código de test

```java
@Test
public void shouldMoveStorm() {
    game.ticTac();
    Storm proofStorm = (Storm) game.get(3, 1);
    assertEquals(proofStorm, sharkNado);
    game.ticTac();
    proofStorm = (Storm) game.get(2, 2);
    assertEquals(proofStorm, sharkNado);
    game.ticTac();
    proofStorm = (Storm) game.get(1, 3); // Hace recorrido
    assertEquals(proofStorm, sharkNado);
    game.ticTac();
    proofStorm = (Storm) game.get(0, 4);
    assertEquals(proofStorm, sharkNado);
    game.ticTac();
    proofStorm = (Storm) game.get(4, 0); // Aqui finaliza en la última y se reinicia
    assertEquals(proofStorm, sharkNado);
}

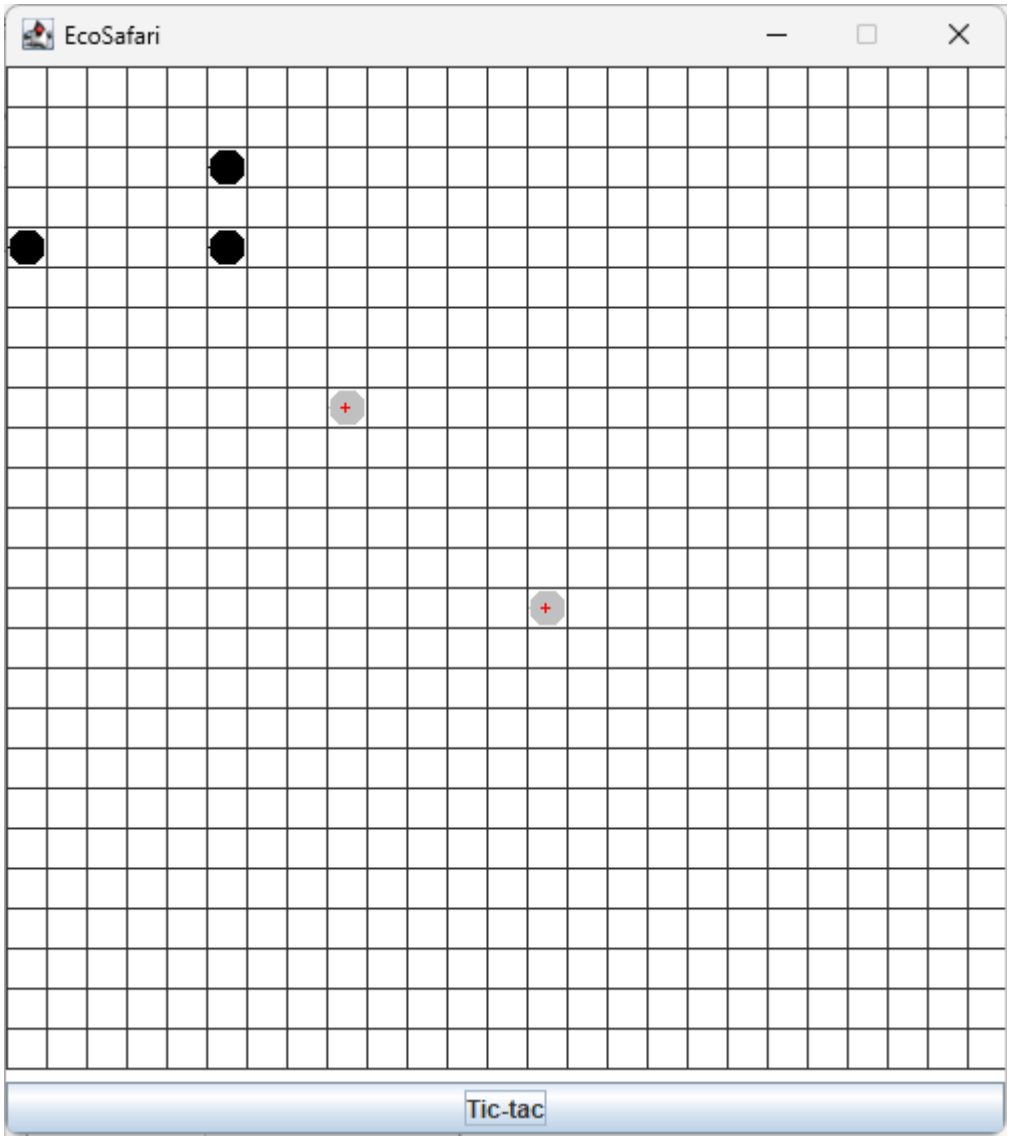
@Test
public void shouldStormDestroyElephant() {
    white =new Elephant(game, 3, 1);
    game.ticTac();
    assertEquals(sharkNado, game.get(3, 1));   // Storm ocupa la celda
    assertNull(game.find(white));               // el elefante ya no está en el tablero
}
```

3. Adicionen dos objetos tormentas en EcoSafari , llámenlos thor y tempest ,

```java
/**
 * Pupulates the EcoSafari with some entities
 */
public void someEntities(){
    Elephant dumbo = new Elephant(this, 5, 5);
    Elephant babar = new Elephant(this, 10, 10);

    Elephant white = new Elephant(this, 3, 1);
    Storm sharkNado = new Storm(this, 4, 0);
    Storm thor  = new Storm(this, 7, 0);
    Storm tempest   = new Storm(this, 9, 0);
}
```

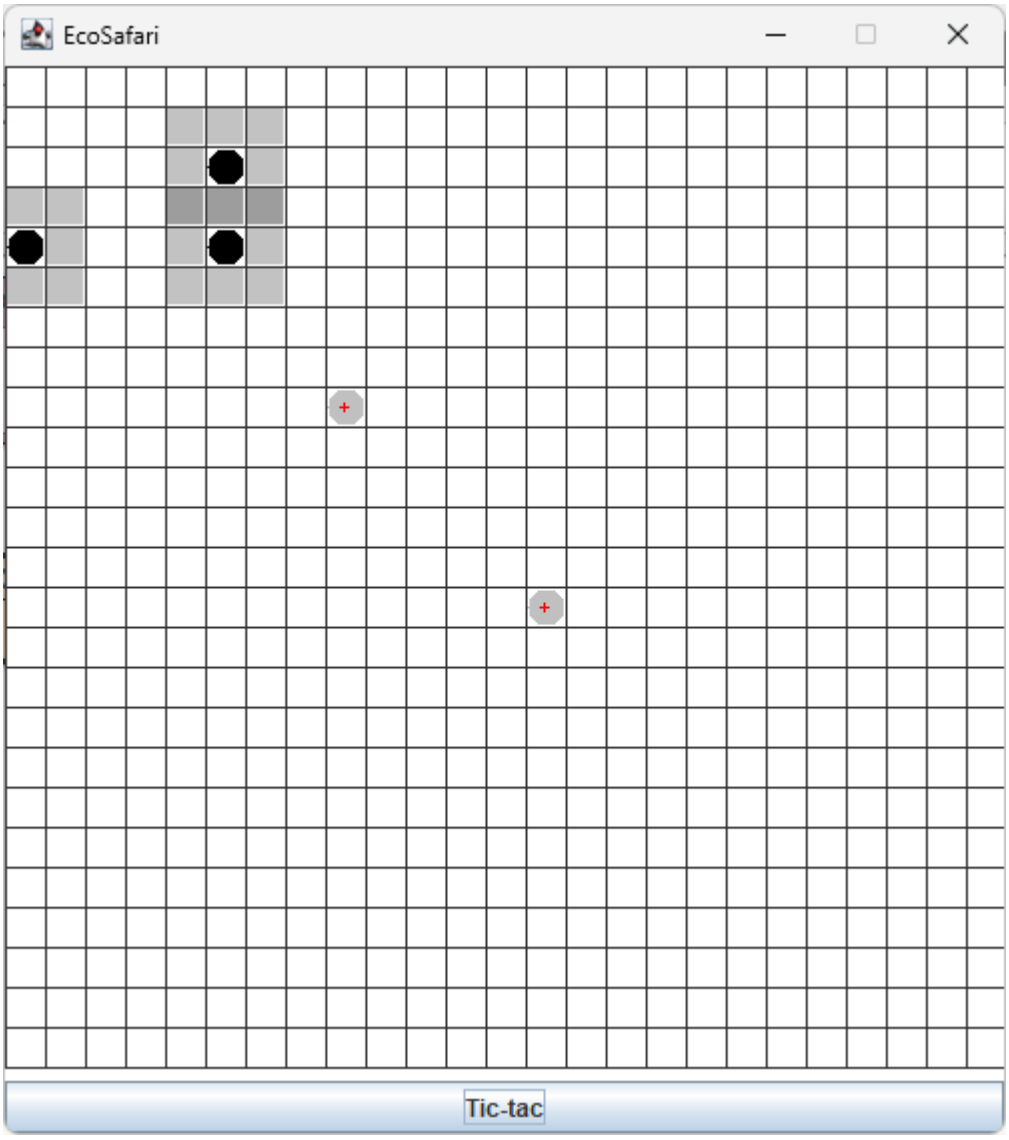
(a) ¿Cómo quedarían después de cuatro Tic-tac? Ejecuten el programa y hagan 5 clics en el botón. Capturen una pantalla significativa.



(b) ¿Es correcto?

La parte del movimiento y donde deben estar y eliminar entidades es correcto, lo que no es correcto es visualmente como se ve la tormenta, con las debidas correcciones en presentación se puede ver de

manera correcta, así como en la siguiente imagen



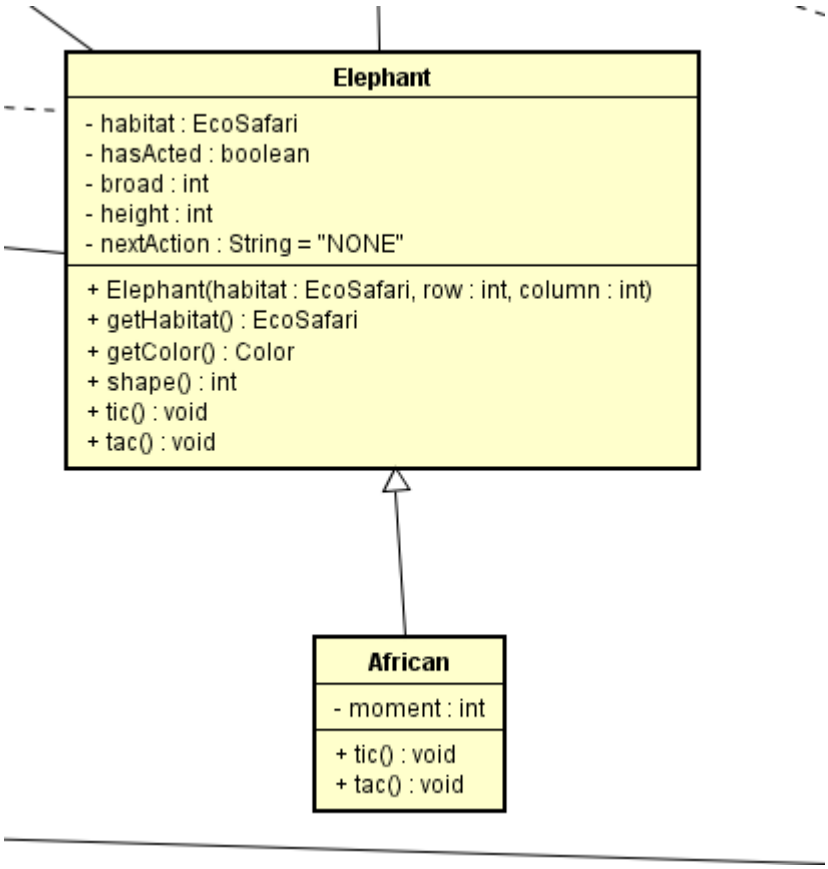
D. Ciclo 4. Proponiendo y diseñando: Nuevo elefante [En lab03.pdf safari.astah *.java]

(NO OLVIDE BDD – MDD)

El objetivo de este punto es permitir recibir en un nuevo tipo de Elephant .

1. Propongan, describan e implementen el nuevo tipo de Elephant . (Mínimo dos pruebas de unidad)

Nuestro nuevo Elephant es Elefante africano, el cual se caracteriza por ser un Elefante perezoso, por lo tanto vamos a simular a este elefante haciendo que se demore el doble en hacer cualquier cosa.



Realizamos las pruebas unitarias:

```java
import domain.EcoSafari;
import domain.African;
import domain.Bush;
import static org.junit.jupiter.api.Assertions.*;
import org.junit.jupiter.api.AfterEach;
import org.junit.jupiter.api.BeforeEach;
import org.junit.jupiter.api.Test;
import java.util.Arrays;
/**
 * The test class TestAfrican.
 *
 * @author  (your name)
 * @version (a version number or a date)
 */
public class TestAfrican
{
    private EcoSafari game;
    private African gualdronn;
    private African steveveenn;

    @Test
    public void shouldTicTac() {
        game = new EcoSafari();
        gualdronn = new African(game,8,8);
        steveveenn = new African(game,15,15);
        int[] initialPosDumbo = game.find(gualdronn);
        int[] initialPosBabar = game.find(steveveenn);
        game.ticTac();
        game.ticTac();
        int[] nextPosDumbo = game.find(gualdronn);
        int[] nextPosBabar = game.find(steveveenn);
        assertFalse(Arrays.equals(initialPosDumbo, nextPosDumbo)); // No deberian estar en la misma posicion
        assertFalse(Arrays.equals(initialPosBabar, nextPosBabar));
    }

    @Test
    public void shouldEat() {
        game = new EcoSafari();
        gualdronn = new African(game,1,1);
        steveveenn = new African(game,15,15);
        Bush bush = new Bush(game, 2, 1);
        Bush bush1 = new Bush(game, 14, 15);
        game.ticTac();
        game.ticTac();
        game.ticTac();
        game.ticTac();
        assertTrue( !(game.get(2, 1) instanceof Bush));
        assertTrue( !(game.get(14, 15) instanceof Bush));
    }
}
```

Hacemos los diagramas de secuencia y programamos:

```java
 */
public class African extends Elephant
{
    private int moment = 0;
    public African(EcoSafari safari, int row, int column) {
        super(safari, row, column);
    }

    /**
     * Determines the color of the elephant based on its current energy level.
     *
     * @return color, LIGHT_GRAY < 80 OR DARK_GRAY >= 80
     */

    @Override
    public void tic(){
        nextAction = "NONE";
        height = habitat.find(this)[0];
        broad = habitat.find(this)[1];
        if ((! hasActed) && habitat.get(height -1,broad) instanceof Bush) {
            nextAction = "EAT";
            height--;

        }
        else if ((! hasActed) && habitat.get(height+1,broad) instanceof Bush) {
            nextAction = "EAT";
            height++;

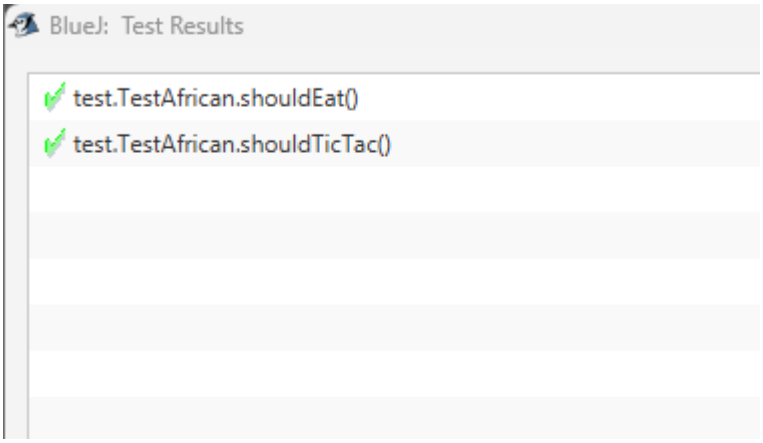
        }
        else if ((! hasActed) && habitat.get(height,broad+1) instanceof Bush) {
            nextAction = "EAT";
            broad++;

        }
        else if ((! hasActed) && habitat.get(height,broad-1) instanceof Bush) {
            nextAction = "EAT";
            broad--;

        }
        else if ((! hasActed) && (move(1, 1))) {
            nextAction = "MOVE";
        }
        hasActed=true;
    }

    @Override
    public void tac(){
        moment++;
        if (moment >= 2) {
            moment = 0;
            if (nextAction.equals("EAT")) {
                changeEnergy(20);
                habitat.set(null,height,broad);
            }
            if (nextAction.equals("MOVE")) {
                changeEnergy(-10);
                if (getEnergy()==0){
                    disappear();
                }
            }
        }
```

Verificamos las pruebas unitarias:



2. Considerando una pareja de ellos con el apellido de ustedes.

```
    Elephant dumbo = new Elephant(this, 5, 5);
    Elephant babar = new Elephant(this, 10, 10);
    Bush mopane  = new Bush(this, 3, 5);
    Bush acacia  = new Bush(this, 8, 5);
    African gualdron = new African(this,8,8);
    African steveveen = new African(this,15,15);
}

/**
 * Returns the size of the EcoSafari
```

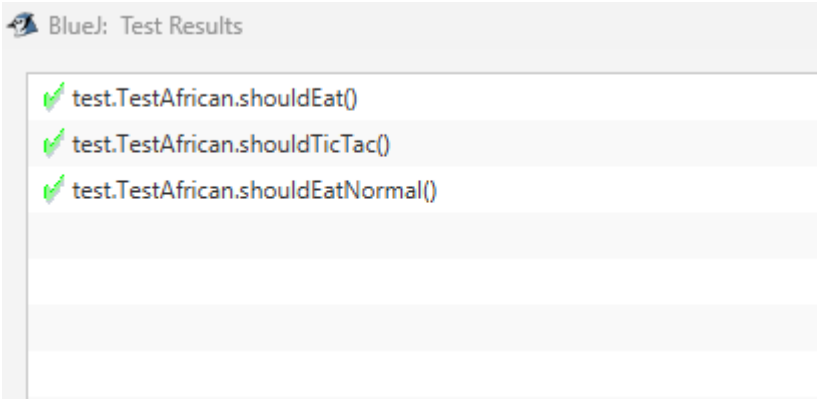
(a) Piensen en otra prueba significativa y expliquen la intención.

La prueba verificara que si se encuentra un elefante normal junto a un arbusto y un elefante perezoso, entonces el elefante normal va a comerse el arbusto y los dos se moveran al mismo tiempo

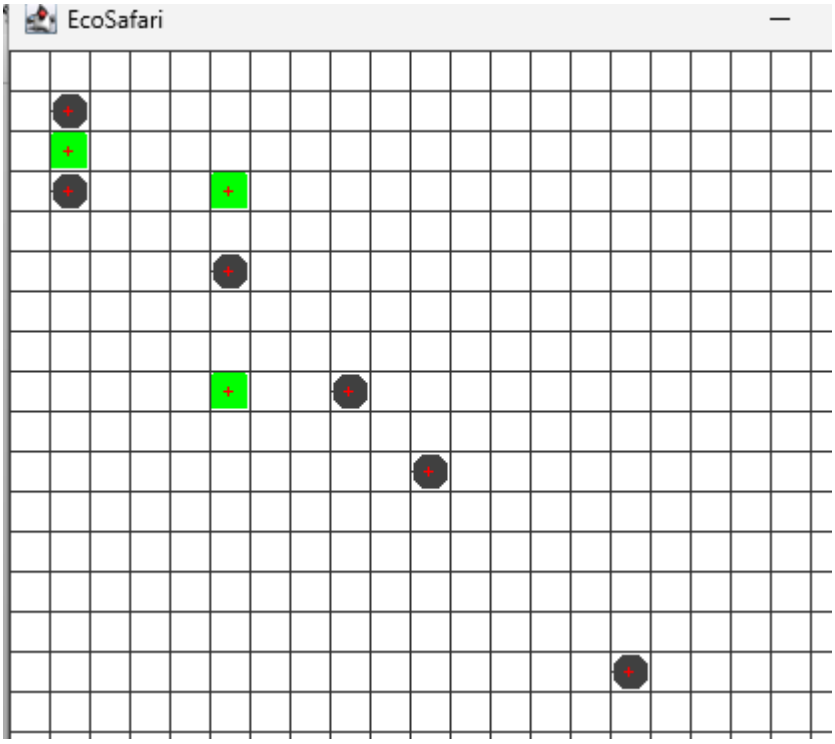
(b) Codifiquen la prueba de unidad correspondiente y capturen la pantalla de resultados de ejecución de la prueba.

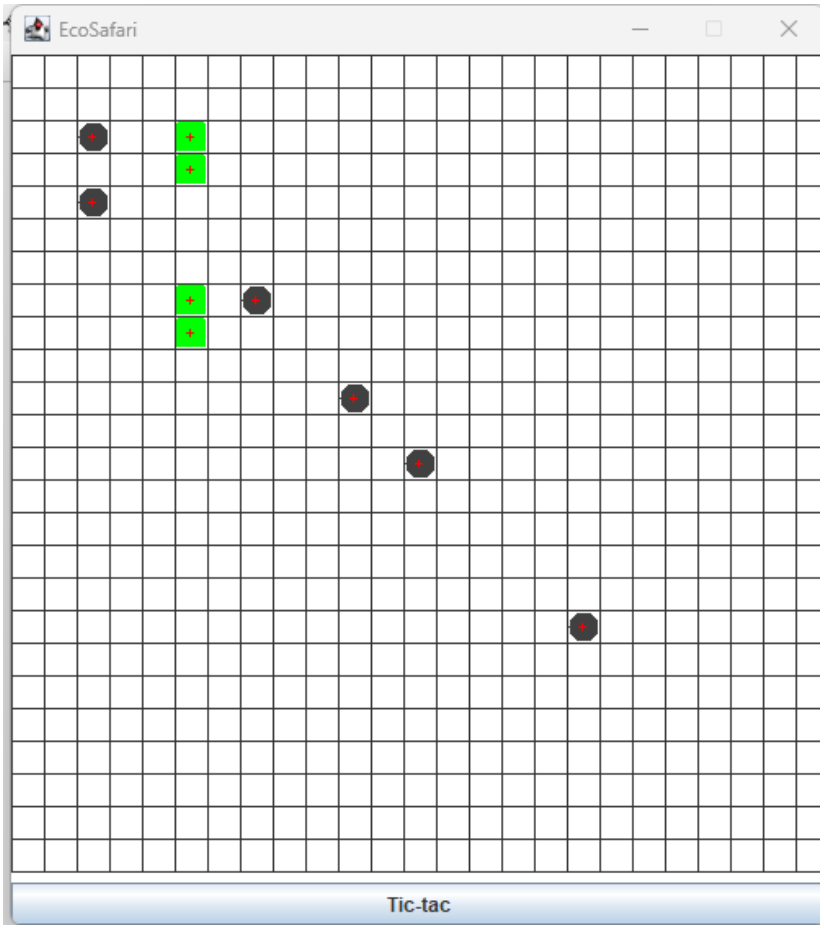
```
    }

    @Test
    public void shouldEatNormal() {
        game = new EcoSafari();
        gualdronn = new African(game,1,1);
        dumbo = new Elephant(game,3,1);
        Bush bush = new Bush(game, 2, 1);
        game.ticTac();
        game.ticTac();
        game.ticTac();
        game.ticTac();
        assertTrue( !(game.get(2, 1) instanceof Bush));
        assertTrue( (game.get(4, 2) instanceof Elephant));
        assertTrue( (game.get(2, 2) instanceof African));
    }
}
```



(c) Ejecuten el programa con esa prueba como prueba de aceptación y capturen las pantallas correspondientes.


EcoSafari


EcoSafari
Tic-tac

E. Ciclo 5. Proponiendo y diseñando : Nueva entidad [En lab03.pdf safari.astah *.java]

(NO OLVIDE BDD – MDD)

El objetivo de este punto es permitir recibir una nueva clase de Entity (no Elephant) en el EcoSafari

1. Propongan, describan e implementen la nueva clase de Entity . (Mínimo dos pruebas de unidad)

Se va a proponer un oso los cuales se cuelan en la sabana por falta de comida en el bosque, este va a ser llamado como Bear, se va a mover un bloque a su alrededor, aleatoriamente; el código de sus pruebas son las siguientes

```
@Test
public void shouldStormDestroyElephant() {
    white =new Elephant(game, 3, 1);
    game.ticTac();
    assertEquals(sharkNado, game.get(3, 1));    // Storm ocupa la celda
    assertNull(game.find(white));                // el elefante ya no está en el tablero
}

@Test
public void shouldBearMakeDamage() {
    EcoSafari game = new EcoSafari();
    Bear bear = new Bear(game, 5, 4);
    Elephant elephant = new Elephant(game, 5, 5);

    for (int i = 0; i < 20; i++) {
        bear.attack();
    }

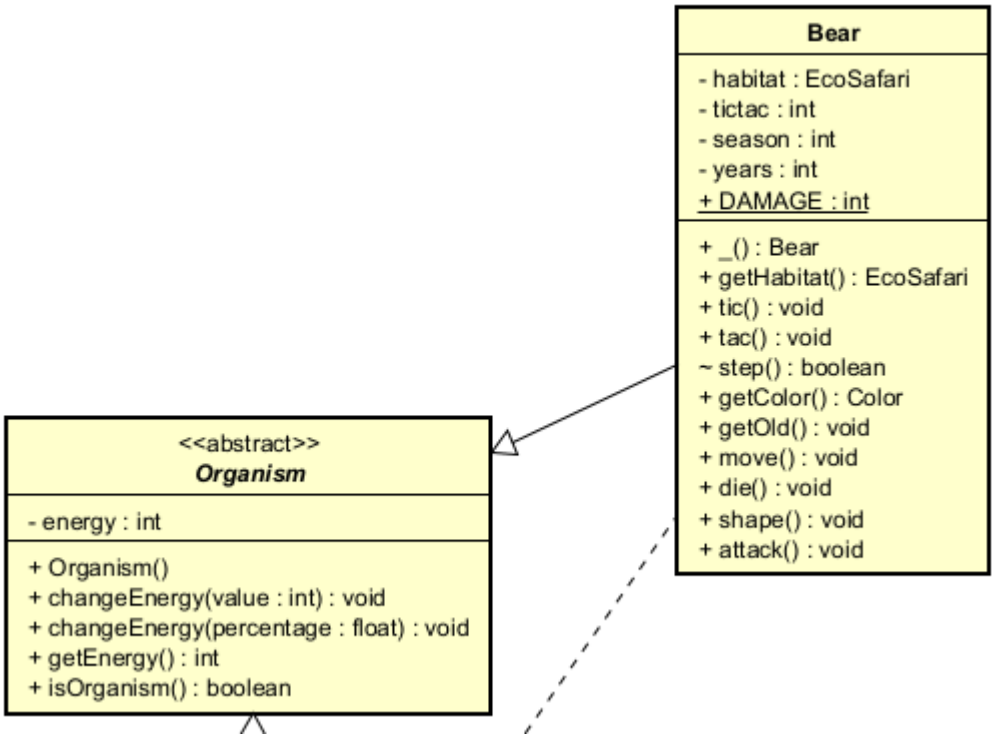
    // 20 ataques * 3 de daño = 60 de daño. 100 - 60 = 40.
    assertEquals(40, elephant.getEnergy());
}

@Test
public void shouldBearMakeDamageBush() {
    EcoSafari game = new EcoSafari();
    Bear bear = new Bear(game, 15, 16);
    Bush bush = new Bush(game, 15, 15);

    for (int i = 0; i < 20; i++) {
        bear.attack();
    }

    assertEquals(40, bush.getEnergy());
}
```

Ahora el diseño es el siguiente



2. Considerando un par de ellos con el nombre de ustedes.

```java
/**
 * Pupulates the EcoSafari with some entities
 */
public void someEntities(){
    Elephant dumbo = new Elephant(this, 5, 5);
    Elephant babar = new Elephant(this, 10, 10);
    Elephant white = new Elephant(this, 3, 1);
    Storm sharkNado = new Storm(this, 4, 0);
    Storm thor  = new Storm(this, 7, 0);
    Storm tempest   = new Storm(this, 9, 0);
    Bush mopane  = new Bush(this, 3, 5);
    Bush acacia  = new Bush(this, 8, 5);

    Bear pablo = new Bear(this, 12, 3);
    Bear samuel = new Bear(this, 18, 9);
}
```

(a) Piensen en otra prueba significativa y expliquen la intención.

```java
@Test
public void shouldBearMoveDifferentPlace() {
    EcoSafari game = new EcoSafari();
    Bear bear = new Bear(game, 15, 16);

    bear.move();

    int[] currentPosition = game.find(bear);
    assertNotNull(currentPosition, "El oso no debería haber desaparecido");

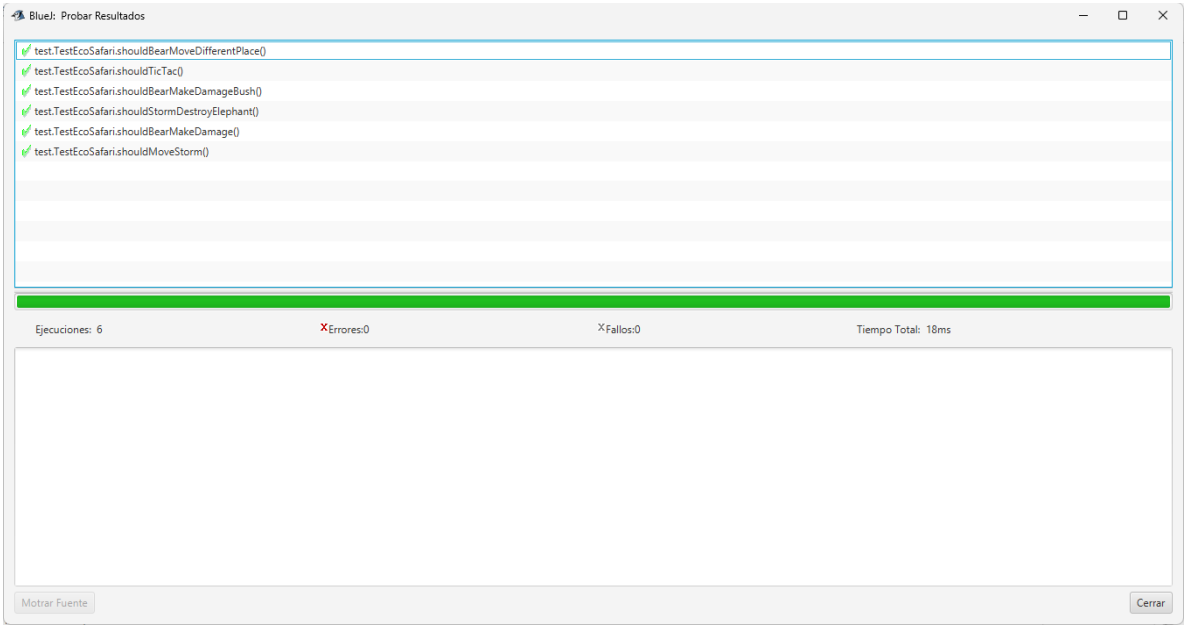
    // Verificamos que haya cambiado de fila o de columna
    boolean movedRow = currentPosition[0] != 15;
    boolean movedCol = currentPosition[1] != 16;

    assertTrue(movedRow || movedCol, "El oso debió cambiar de posición");
}
```

Esta define el movimiento que ha de tener el oso

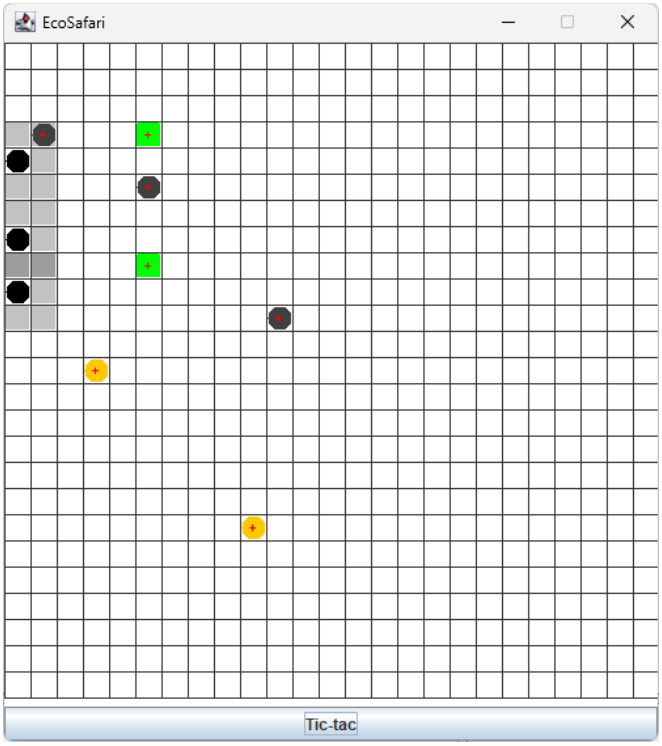
(b) Codifiquen la prueba de unidad correspondiente y capturen la pantalla de resultados de ejecución de la prueba.

Aquí se muestra que pasaron las pruebas de unidad

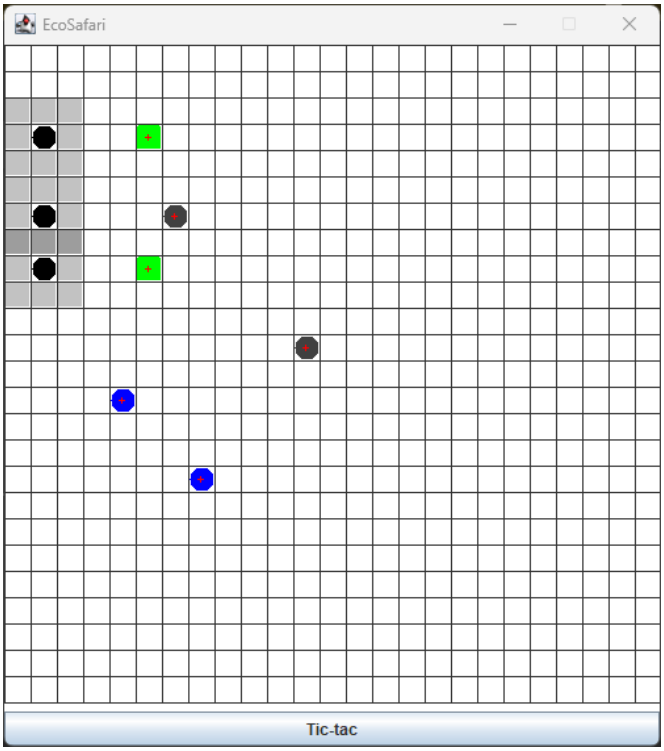


(c) Ejecuten el programa con esa prueba como prueba de aceptación y capturen las pantallas correspondientes.

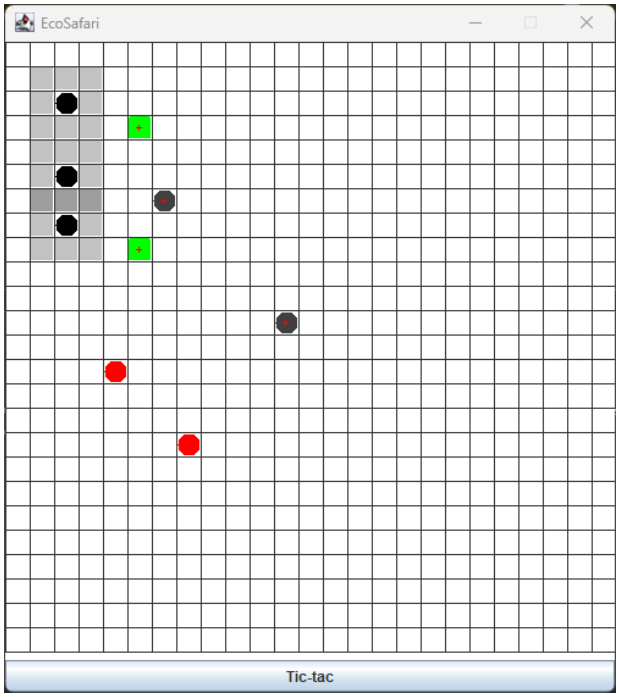
Al principio inicia así el Safari



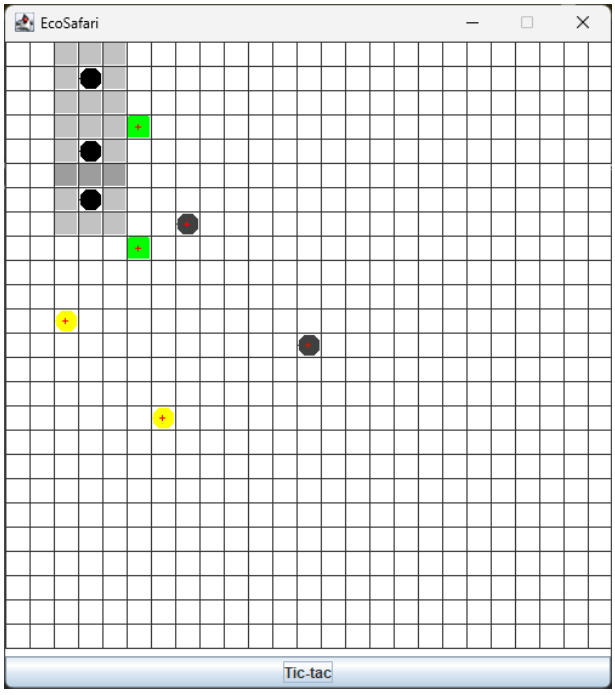
Después de un tictac



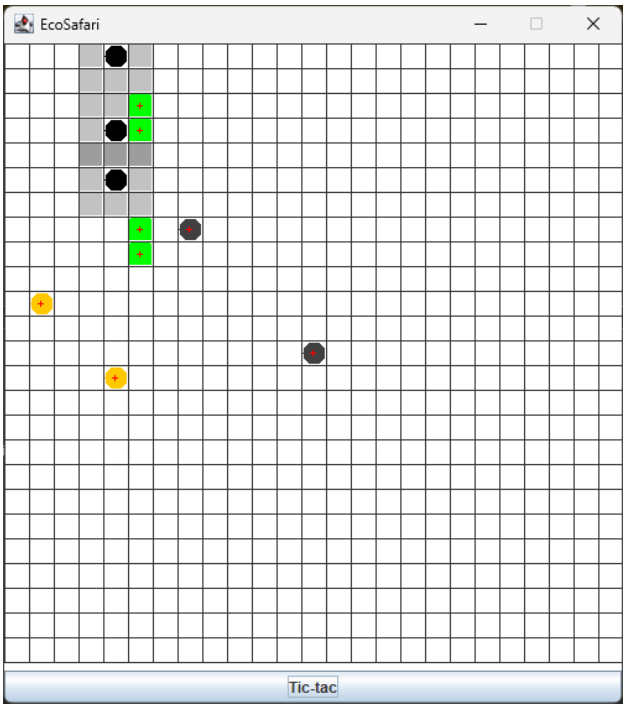
Cuando está en azul es porque está el oso en etapa de hibernar y no se mueve ni ataca, después de otro tictac se muestra lo siguiente



Siguiente Tic-tac



Siguiente Tic-tac



Y se observa que se tienen los 4 colores diferentes de Bear

PARTE III. Desarrollando el reto [En lab03.pdf safari.astah *.java]

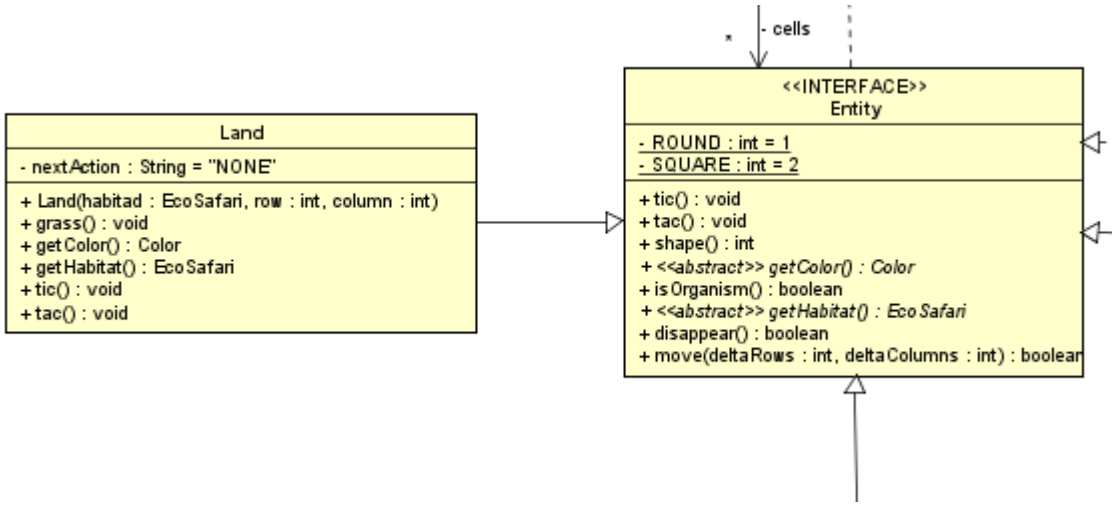
(NO OLVIDE BDD – MDD)

En este punto crearemos una zona en la pradera de **EcoSafari** dedicada a examinar lo que ocurre siguiendo un modelo depredador-presa-recurso que se comporta con las siguientes reglas:

- Cada celda tiene ocho celdas vecinas (vecindario de Moore).
- Las únicas entidades de la zona de la pradera son: tierra, pasto, cebra y león.
- Los animales se mueven libremente sobre tierra. Los leones avanzan una celda por paso y las zebras dos. Cada vez que se desplazan, pierden 10 % de su energía e intercambian su posición con la celda de tierra ocupada.
- Si la cebra queda vecina al pasto, se lo come y aumenta su energía en un 25 %.
- Si el león queda vecino a una cebra, se la come y aumenta en un 50 % su energía.
- Si un animal queda a una casilla de tierra de otro de su misma especie, en la celda de tierra nace un nuevo animal de esa especie.
- Las celdas de tierra pueden generar pasto con una probabilidad del 10 %.
- Los animales no pueden tener más de 100 % de energía y mueren cuando se quedan sin energía. Cuando alguna entidad desaparece, es reemplazada por tierra.

Todo sobre Land:

EL diagrama de clase de Land es:



Estructura base de la clase:

```java
package domain;
import java.awt.Color;

/**
 * Write a description of class Land here.
 *
 * @author (your name)
 * @version (a version number or a date)
 */
public class Land implements Entity
{
    Color brown = new Color(139, 69, 19);
    protected final EcoSafari habitat;
    protected boolean hasActed;

    public Land(EcoSafari habitat,int row, int column) {
        this.habitat = habitat;
    }

    public EcoSafari getHabitat(){
        return habitat;
    }

    public final Color getColor(){
        return brown;
    }

    public void tic(){

    }

    public void tac(){


        hasActed=false;
    }
}
```

Se ajusta la creación para que todo sea tierra al inicio:

```java
/**
 * Pupulates the EcoSafari with some entities
 */
public void someEntities(){
    for (int r = 0; r <= SIZE; r++) {
        for (int m = 0; m <= SIZE; m++) {
            new Land(this,r,m);
        }
    }
}
```

Planteamos las pruebas unitarias:

Compile | Undo | Cut | Copy | Paste | Find... | Close

```java
package test;

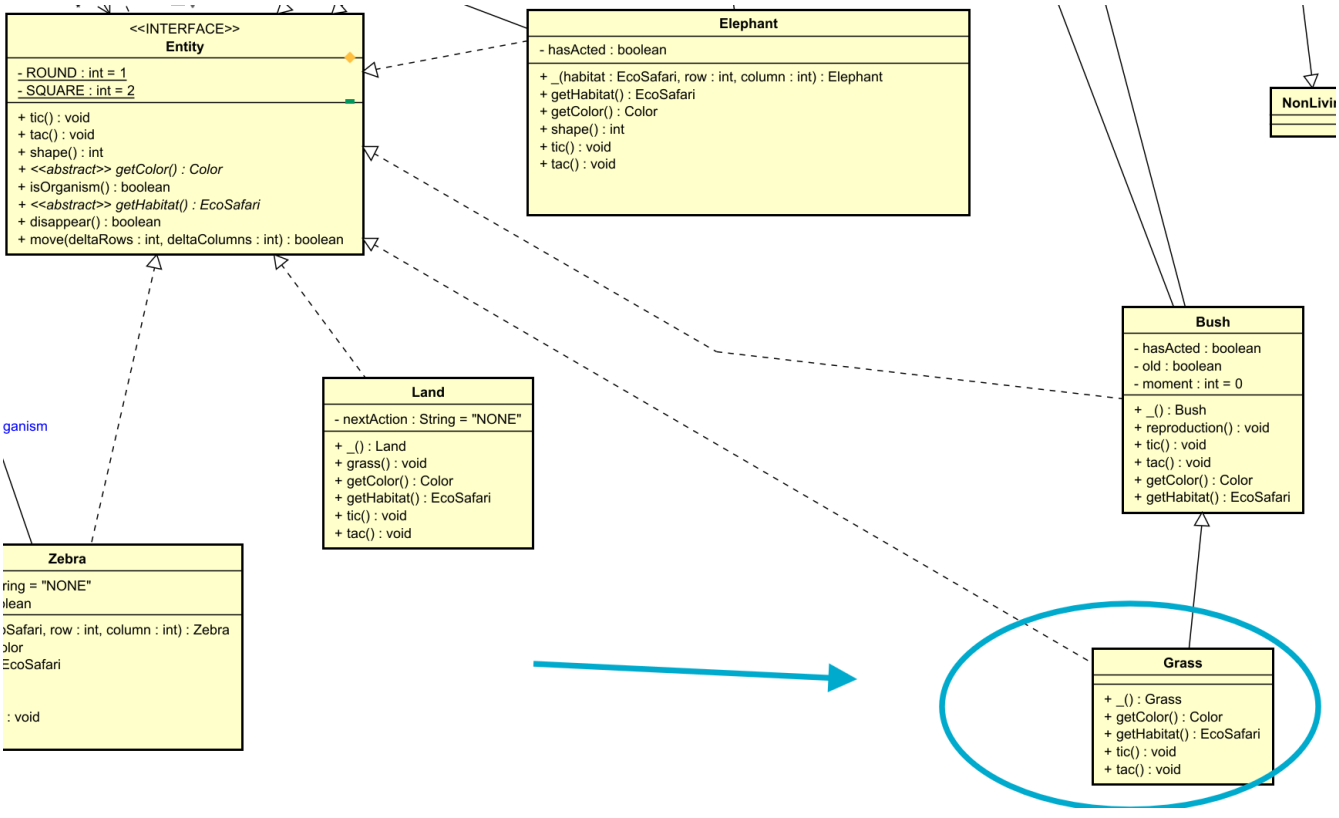

import domain.Land;
import domain.Grass;
import domain.EcoSafari;
import static org.junit.jupiter.api.Assertions.*;
import org.junit.jupiter.api.AfterEach;
import org.junit.jupiter.api.BeforeEach;
import org.junit.jupiter.api.Test;
import org.junit.Before;

/**
 * The test class TestLand.
 *
 * @author  (your name)
 * @version (a version number or a date)
 */
public class TestLand
{
    private EcoSafari habitat;

    @Before
    public void setUp() {
        habitat = new EcoSafari();
    }

    @Test
    public void shouldGrass() {
        for (int i = 0; i <= 10; i++){
            habitat.ticTac();
        }
        for (int i = 0; i <= 25; i++){
            for (int u = 0; u <= 25; u++){
                assertTrue(habitat.get(i,u) instanceof Grass);
            }
        }
    }
}
```

Todo sobre Grass:



Código base de Grass

```java
package domain;
import java.awt.Color;
import java.util.Random;

public class Grass extends Bush implements Entity {

    public Grass(EcoSafari habitat, int row, int column) {
        super(habitat, row, column);
        habitat.set(this, row, column);
    }

    @Override
    public Color getColor() {
        return Color.GREEN;
    }

    @Override
    public EcoSafari getHabitat() {
        return habitat;
    }

    @Override
    public void tic() {
        // El pasto no realiza acciones en tic
    }

    @Override
    public void tac() {
        // El pasto no realiza acciones en tac
    }

}
```

Todo de Lion:

PARTE IV. Empaquetando el proyecto [En lab03.pdf safari.jar]

En un JAR (Java ARchive) se pueden empaquetar múltiples clases y archivos para distribuir y ejecutar artefactos software en un solo archivo comprimido.

1. Revise las opciones de BlueJ para empaquetar su programa en un archivo .jar. Genere el archivo correspondiente.
   (a) Incluya una pantalla con evidencia de la existencia del archivo.

2. Consulte el comando java para ejecutar un archivo jar. ejecútenlo
   (a) ¿Qué pasa?

   (b) ¿Qué ventajas tiene esta forma de entregar los proyectos? Explique claramente.

RETROSPECTIVA

¿Cuál fue el tiempo total invertido en el laboratorio por cada uno de ustedes? (Horas)

Los dos dedicamos 20 horas al laboratorio

1. ¿Cuál es el estado actual del laboratorio? ¿Por qué?

Es incompleto; no se alcanzó a cumplir con todos los requerimientos de Lion.

2. Considerando las prácticas XP del laboratorio, ¿cuál fue la más útil? ¿por qué?

Unit Test First porque esta es importante al tener un sustento de pruebas para comprobar lo que se hizo como implementaciones de métodos y clases.

3. ¿Cuál consideran fue el mayor logro? ¿Por qué?

El mayor logro es cumplir con casi todos los requerimientos.

4. ¿Cuál consideran que fue el mayor problema técnico? ¿Qué hicieron para resolverlo?

Github dio problemas en el caso de que se borraron archivos importantes de Java y compartimos entre nosostros el contenido para tener todo.

5. ¿Qué hicieron bien como equipo? ¿Qué se comprometen a hacer para mejorar los resultados?

La comunicación.

6. ¿Qué referencias usaron? ¿Cuál fue la más útil? Incluyan citas con estándares adecuados.

Oracle. (s.f.). *ClassCastException (Java SE 21 & JDK 21)*. Oracle Help Center. oracle.comOracle. (s.f.). *Abstract Methods and Classes*. The Java Tutorials. Oracle.com

JUnit Team. (s.f.). *Assertions*. GitHub Wiki. github.com